

计算机组成原理

Computer Organization & Architecture

MS Roslyn

2026 年 10 月 9 日

摘要

本教程系统讲解计算机组成原理的核心概念，从底层硬件到上层接口逐层展开：数据在机器中的表示与运算、数字逻辑基础、指令系统设计、处理器数据通路与控制单元、流水线与指令级并行、存储层次与缓存、I/O 系统与总线架构。面向计算机专业学生及自学者，兼顾理论基础与工程直觉。

目录

1	计算机系统概述	7
1.1	计算机的发展与分类	8
1.1.1	摩尔定律	8
1.2	冯·诺依曼体系结构	9
1.2.1	五大部件与总线连接	9
1.2.2	存储程序思想	10
1.2.3	指令执行周期	10
1.3	计算机层次结构与软硬件接口	11
1.4	性能评价	12
1.5	Amdahl 定律	13
1.6	基本概念：字长、主频、时钟周期与 CPI	14
1.7	本章小结与常见误区	14
2	数据表示与运算	16
2.1	数制与编码	17
2.2	整数表示	17
2.2.1	无符号整数	17
2.2.2	原码 (Sign-Magnitude)	17
2.2.3	补码 (Two's Complement)	17
2.2.4	移码 (Biased Notation)	18
2.3	整数运算	18
2.3.1	加法器	18
2.3.2	溢出检测	18
2.4	浮点数表示：IEEE 754	19
2.5	浮点运算	20
2.6	字符编码	20
3	数字逻辑基础	21
3.1	CMOS 元件与门电路内部结构	22
3.1.1	MOSFET 开关模型	22
3.1.2	CMOS 反相器 (NOT)	22
3.1.3	CMOS NAND (与非门)	23
3.1.4	由 NAND 派生其它门	23
3.2	布尔代数与门电路	24
3.3	逻辑表达式化简	25

3.3.1	代数化简	25
3.3.2	卡诺图 (Karnaugh Map)	25
3.4	组合逻辑电路：加法器	25
3.4.1	半加器 (Half Adder)	25
3.4.2	全加器 (Full Adder)	26
3.4.3	行波进位加法器 (Ripple Carry Adder)	26
3.4.4	超前进位加法器 (Carry Lookahead Adder)	27
3.4.5	溢出检测	27
3.5	组合逻辑电路：数据选择器与译码器	28
3.5.1	多路选择器 (Multiplexer, MUX)	28
3.5.2	译码器 (Decoder)	29
3.6	算术逻辑单元 (ALU)	30
3.7	时序逻辑电路	31
3.7.1	SR 锁存器	31
3.7.2	D 触发器 (D Flip-Flop)	32
3.7.3	n 位寄存器	32
3.8	有限状态机 (FSM)	33
4	指令系统与汇编	34
4.1	指令系统：软件与硬件的接口	35
4.1.1	ISA 在分层体系中的位置	35
4.1.2	ISA 规定的内容	35
4.2	指令格式	36
4.2.1	指令的组成	36
4.2.2	三种指令格式的字段划分	37
4.3	寻址方式	38
4.3.1	各种寻址方式的有效地址计算式	38
4.4	MIPS 常用指令分类	39
4.4.1	示例机器码	40
4.4.2	完整例题：把汇编翻译成机器码	40
4.5	CISC 与 RISC	41
4.6	过程调用与栈帧	42
4.6.1	调用约定	42
4.6.2	栈帧结构	43
4.6.3	示例：数组求和	44
4.7	本章小结	44

4.8	常见误区	45
5	处理器：数据通路	46
5.1	数据通路的组成部件	47
5.2	单周期执行的五个阶段	48
5.3	四类指令的数据流	49
5.3.1	R 型指令 (add/sub/and/or/slt)	49
5.3.2	I 型: lw (取字)	49
5.3.3	I 型: sw (存字)	50
5.3.4	I 型: beq (相等则分支)	50
5.4	控制信号在数据通路中的位置	51
5.5	关键路径与性能	52
5.6	本章小结	53
5.7	常见误区	54
6	处理器：控制器	55
6.1	控制器的作用与模型	56
6.2	控制信号的语义与取值	57
6.2.1	主要控制信号	57
6.2.2	ALU 控制器: ALUOp 与 funct 到 ALUctr	58
6.2.3	ALUctr 的化简	58
6.3	典型指令的控制信号组合	59
6.4	硬布线控制器	60
6.4.1	从真值表到布尔表达式	60
6.4.2	化简例题	60
6.5	微程序控制器	61
6.5.1	微指令格式与控制存储器	61
6.5.2	微程序的执行	62
6.5.3	与硬布线的对比	62
6.6	单周期与多周期 CPU	63
6.6.1	状态划分	63
6.6.2	各状态的控制信号	64
6.6.3	为什么多周期的时钟周期更短	64
6.7	本章小结	65
6.8	常见误区	65

7	流水线与指令级并行	66
7.1	流水线思想与时空图	67
7.2	理想加速比与吞吐率	68
7.3	经典五级流水线	69
7.3.1	五个阶段	69
7.3.2	流水线寄存器的作用	69
7.3.3	时空图	70
7.4	流水线冒险	70
7.4.1	结构冒险	70
7.4.2	数据冒险与数据前递	70
7.4.3	控制冒险	71
7.5	停顿、分支惩罚与分支预测	72
7.5.1	停顿与分支延迟槽	72
7.5.2	静态预测	72
7.5.3	动态预测	73
7.6	流水线性能分析	74
7.7	超标量与乱序执行	75
7.8	本章小结与常见误区	76
8	存储层次与缓存	77
8.1	存储层次概述	78
8.2	局部性原理	78
8.3	SDRAM 与 DRAM	78
8.4	缓存 (Cache)	79
8.4.1	直接映射缓存 (Direct-Mapped)	79
8.4.2	组相联缓存 (Set-Associative)	79
8.4.3	缓存地址划分	79
8.4.4	缓存缺失与写策略	80
8.4.5	缓存性能	80
8.5	虚拟内存	80
8.5.1	TLB (快表)	80
8.5.2	多级页表	80
8.6	存储层次设计原则	81
9	I/O 系统与总线	82
9.1	I/O 系统的层次	83

9.2	内存映射 I/O vs 独立 I/O	83
9.3	I/O 同步方式	83
9.3.1	轮询 (Polling)	83
9.3.2	中断 (Interrupt)	83
9.3.3	直接存储器访问 (DMA)	84
9.4	总线	84
9.4.1	总线分类	84
9.4.2	现代总线: PCIe	85
9.5	磁盘与固态存储	85
9.6	多核与并行 I/O	85
9.7	系统总线架构总结	86

1 计算机系统概述

1.1 计算机的发展与分类

计算机（Computer）是按预定程序自动、高速地对信息进行数值计算与逻辑处理的电子设备。自 1946 年 ENIAC 问世以来，其发展大体经历了四个阶段，如表 1 所示。

表 1: 计算机发展的主要分代

分代	时期	核心器件	典型代表
第一代	1946–1957	电子管	ENIAC、EDVAC
第二代	1958–1964	晶体管	IBM 7094
第三代	1965–1971	中小规模集成电路（SSI/MSI）	IBM System/360
第四代	1971 至今	大规模/超大规模集成电路（LSI/VLSI）	微处理器（Intel 4004 起）

按用途可分为通用计算机与专用计算机；按规模与形态可分为巨型机、大型机、小型机、工作站、微型机（PC）与嵌入式系统。分类的边界随技术演进而不断模糊——今天的手机算力已超过昔日的大型机，因此更应关注其体系结构共性而非名义类别。

1.1.1 摩尔定律

定义 1.1 (摩尔定律). 集成电路上可容纳的晶体管数目约每 18–24 个月翻一番，而单位成本基本不变。

摩尔定律并非物理定律，而是对产业趋势的经验观察。它解释了为什么在价格不变的前提下算力持续攀升，也预示其终将遭遇物理极限（功耗墙、散热瓶颈、量子隧穿）。对学习组成原理的启示是：硬件的“免费午餐”终有尽头，性能提升必须回到体系结构本身——更聪明的流水线、更深的存储层次、更好的并行。

1.2 冯·诺依曼体系结构

定义 1.2 (冯·诺依曼体系结构). 计算机由五大部件构成:

- 运算器 (*Arithmetic Logic Unit, ALU*): 执行算术与逻辑运算
- 控制器 (*Control Unit, CU*): 取指令、译码、发出控制信号
- 存储器 (*Memory*): 存放程序与数据
- 输入设备 (*Input*): 键盘、鼠标、传感器等
- 输出设备 (*Output*): 显示器、打印机、网络接口等

运算器与控制器合称中央处理器 (*CPU*); 程序与数据统一以二进制形式存储在同一存储器中——这是冯·诺依曼体系的核心特征: 存储程序 (*Stored Program*)。

$$\text{计算机} = \text{CPU} + \text{存储器} + \text{I/O} \quad (1)$$

1.2.1 五大部件与总线连接

五大部件通过系统总线 (数据总线、地址总线、控制总线) 互连, 如图 1 所示。控制器发出控制信号协调各部件, 存储器既是数据仓库也是程序仓库。

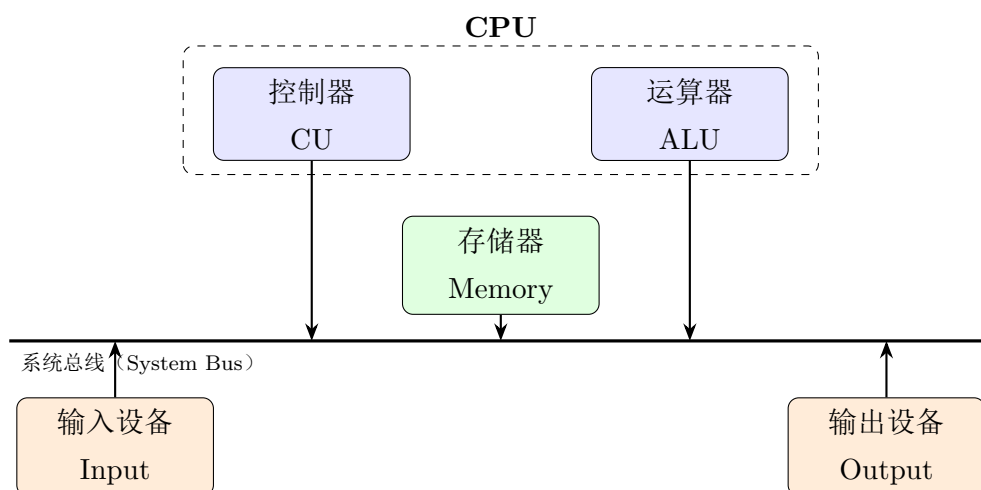


图 1: 冯·诺依曼结构的五大部件与系统总线

1.2.2 存储程序思想

冯·诺依曼模型的关键在于指令和数据存放在同一个存储器中，CPU 按顺序逐条取出并执行。这一设计使得计算机成为“通用机器”——更换程序即可完成不同任务，无需改动硬件。由此引申出三条要点：

- 程序与数据同构：二者都是二进制位串，由取用时机和语境区分。
- 顺序执行：默认按地址递增取指，除非遇到分支/跳转。
- 指令驱动：硬件不做主动决策，一切行为由指令序列决定。

1.2.3 指令执行周期

每条指令的执行可分解为四个基本阶段，构成不断循环的“冯·诺依曼循环”，如图 2 所示。

Fetch → Decode → Execute → Write-back

取指：PC → MAR, $M[MAR] \rightarrow MDR$

译码：CU 解析操作码与地址码 (2)

执行：ALU 运算 / 访存 / 计算跳转地址

写回：结果写回寄存器或存储器

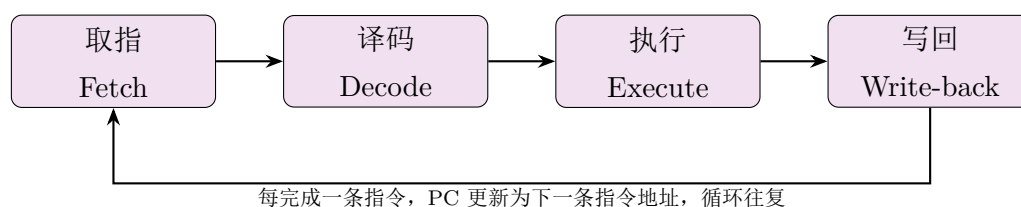


图 2: 指令执行周期：取指 → 译码 → 执行 → 写回

1.3 计算机层次结构与软硬件接口

现代计算机是一个层次化系统，每一层对上一层隐藏细节，如图 3 所示。

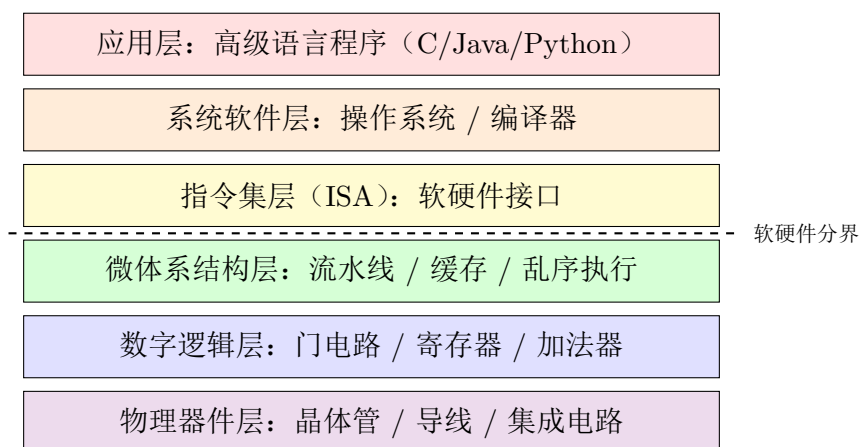


图 3: 计算机系统的层次结构：自上而下逐层抽象

- 应用层：高级语言程序 (C/Java/Python)
- 系统软件层：编译器、操作系统
- 指令集层：ISA——软件与硬件的接口
- 微体系结构层：处理器实现（流水线、缓存、乱序执行）
- 数字逻辑层：门电路、寄存器、加法器
- 物理器件层：晶体管、导线、集成电路

指令集体系结构 (ISA) 是软硬件之间的契约：它规定程序员可见的指令、寄存器与寻址方式，却不限定内部如何实现。同一 ISA 可以由不同的微体系结构实现（如 x86 由 Intel 与 AMD 分别实现），而上层软件无需改动即可运行。数据在各层之间的“翻译”链条为：

高级语言 → 机器指令 (ISA) → 微体系结构 → 逻辑门 → 物理器件 (3)

“计算机组成原理”关注的是**指令集层到数字逻辑层之间的内容**——硬件如何执行指令、如何在速度、成本与功耗之间权衡。

1.4 性能评价

评价计算机性能时，最客观的指标是运行某段程序的 **CPU 时间**，而非单一部件指标。设程序执行的指令条数为 IC (Instruction Count)，平均每条指令的时钟周期数为 CPI (Cycles Per Instruction)，时钟周期为 T_{clk} ，主频为 $f = 1/T_{clk}$ ，则：

$$T_{CPU} = IC \times CPI \times T_{clk} = \frac{IC \times CPI}{f} \quad (4)$$

由该式可见，提升性能只有三条路径：**减少指令数**（更好的 ISA/编译器）、**降低 CPI**（更好的微体系结构）、**提高主频**（更好的电路与工艺）。吞吐量指标 **MIPS** (Million Instructions Per Second) 定义为：

$$MIPS = \frac{f}{CPI \times 10^6} \quad (5)$$

表 2: 性能评价核心指标

指标	公式	说明
时钟周期	$T_{clk} = 1/f$	主频的倒数，基本时间粒度
CPU 时间	$T_{CPU} = IC \times CPI \times T_{clk}$	程序运行总时间（越低越好）
MIPS	$MIPS = \frac{f}{CPI \times 10^6}$	每秒百万条指令（越高越好）
Amdahl 加速比	$S = \frac{1}{(1-p) + p/n}$	并行/局部优化的加速上限

示例 1.1 (由 IC 、 CPI 、 f 求 CPU 时间与 MIPS). 某程序共执行 $IC = 1 \times 10^9$ 条指令，平均 $CPI = 2.5$ ，处理器主频 $f = 2\text{ GHz}$ 。求该程序的 CPU 时间与 MIPS。

解：时钟周期为

$$T_{clk} = \frac{1}{f} = \frac{1}{2 \times 10^9} = 0.5\text{ ns} \quad (6)$$

CPU 时间为

$$T_{CPU} = IC \times CPI \times T_{clk} = 1 \times 10^9 \times 2.5 \times 0.5 \times 10^{-9} = 1.25\text{ s} \quad (7)$$

用频率形式验算： $T_{CPU} = \frac{IC \times CPI}{f} = \frac{10^9 \times 2.5}{2 \times 10^9} = 1.25\text{ s}$ ，结果一致。MIPS 为

$$MIPS = \frac{f}{CPI \times 10^6} = \frac{2 \times 10^9}{2.5 \times 10^6} = 800 \quad (8)$$

即该处理器每秒执行约 8 亿条指令。

1.5 Amdahl 定律

对系统做局部优化时，整体加速受不可优化部分的制约，这就是 Amdahl 定律。

设优化前程序总时间为 T ，其中可优化部分占比为 p ，不可优化部分占比为 $1 - p$ 。优化后该部分加速 n 倍，则新时间为

$$T_{\text{new}} = (1 - p)T + \frac{p}{n}T = T \left[(1 - p) + \frac{p}{n} \right] \quad (9)$$

故整体加速比为

$$S = \frac{T}{T_{\text{new}}} = \frac{1}{(1 - p) + \frac{p}{n}} \quad (10)$$

当 $n \rightarrow \infty$ 时 $p/n \rightarrow 0$ ，得到加速上限

$$S_{\text{max}} = \lim_{n \rightarrow \infty} S = \frac{1}{1 - p} \quad (11)$$

换言之，无论怎样加速可优化部分，整体加速比都无法突破 $1/(1 - p)$ ——“串行部分是并行加速的天花板”。

示例 1.2 (Amdahl 定律的加速比与上限). 某程序中可并行化部分占 $p = 80\%$ ，用 $n = 4$ 个处理器加速该部分。求整体加速比与理论加速上限。

解：代入公式

$$S = \frac{1}{(1 - p) + p/n} = \frac{1}{(1 - 0.8) + 0.8/4} = \frac{1}{0.2 + 0.2} = 2.5 \quad (12)$$

即 4 个处理器仅带来 2.5 倍整体加速（远小于 4）。当处理器数趋于无穷时

$$S_{\text{max}} = \frac{1}{1 - p} = \frac{1}{0.2} = 5 \quad (13)$$

即便使用无穷多个处理器，整体最多也只能加速 5 倍。可见把 p 从 80% 提升到 90% 比堆叠更多处理器更有效。

1.6 基本概念：字长、主频、时钟周期与 CPI

定义 1.3 (机器字长). CPU 中通用寄存器（或数据通路）的位数，通常等于一次能并行处理、运算的二进制位数。字长决定机器的寻址空间、运算精度与数据表示范围（如 32 位机、64 位机）。

- **主频 f** : CPU 时钟每秒振荡的次数，单位 Hz。 f 越高，单位时间内的时钟节拍越多。
- **时钟周期 T_{clk}** : 一个时钟节拍的时间， $T_{clk} = 1/f$ ，是 CPU 工作的最小时间单位。
- **CPI**: 执行一条指令平均所需的时钟周期数， $CPI = \frac{\text{总时钟周期数}}{IC}$ ；它取决于指令混合与微体系结构，并非常数。

三者与指令数共同决定 CPU 时间，关系可归纳为：

$$T_{CPU} = \frac{IC \times CPI}{f} \iff CPI = \frac{T_{CPU} \times f}{IC} \quad (14)$$

表 3: 基本概念一览

概念	符号	关系/含义
机器字长	W	一次并行处理的位数，决定精度与寻址
主频	f	每秒时钟次数，单位 Hz
时钟周期	T_{clk}	$T_{clk} = 1/f$ ，最小时间单位
每条指令周期数	CPI	总周期数 / 指令数，随程序变化

1.7 本章小结与常见误区

本章从冯·诺依曼体系出发，勾勒了计算机的部件构成、层次结构与性能评价方法，核心结论如下：

- 冯·诺依曼体系 = 五大部件 + 存储程序，程序与数据同存于内存。
- 指令执行周期为 取指 $\rightarrow$ 译码 $\rightarrow$ 执行 $\rightarrow$ 写回，由 PC 驱动循环。
- 计算机是分层的，ISA 是软硬件的接口，微体系结构负责实现 ISA。
- 性能的核心指标是 CPU 时间 $T_{CPU} = IC \times CPI / f$ ，MIPS 是吞吐量指标。
- Amdahl 定律表明局部优化的整体收益受串行部分限制，上限为 $1/(1-p)$ 。

初学者常犯的错误汇总于表 4。

表 4: 常见误区与正解

误区	正解
主频越高，程序一定越快	还取决于 CPI 与指令数；不同 ISA 之间不能直接比主频
MIPS 越大性能越好	MIPS 只在同一 ISA、同一程序下才可比较
增加处理器/核数可无限加速	受 Amdahl 上限 $1/(1-p)$ 约束，串行部分决定天花板
CPI 是处理器固定常数	随指令混合与微体系结构而变，是平均值
指令和数据是两种不同的存储	二者同存于内存，仅由使用方式区分（存储程序思想）

2 数据表示与运算

2.1 数制与编码

计算机中所有信息都以二进制（0/1）存储。本章讨论如何用二进制表示整数、浮点数、字符。

表 5: 常用数制对比

进制	基数	数字符号
二进制 (Binary)	2	0, 1
八进制 (Octal)	8	0-7
十进制 (Decimal)	10	0-9
十六进制 (Hex)	16	0-9, A-F

$$(N)_r = \sum_{i=-m}^{n-1} d_i \cdot r^i \quad (15)$$

2.2 整数表示

2.2.1 无符号整数

$$U(X) = \sum_{i=0}^{n-1} x_i \cdot 2^i, \quad \text{范围: } [0, 2^n - 1] \quad (16)$$

2.2.2 原码 (Sign-Magnitude)

最高位为符号位（0 正 / 1 负），其余位表示绝对值。存在 +0（0000）和 -0（1000）两个零。

2.2.3 补码 (Two's Complement)

现代计算机统一采用补码表示有符号整数——优势是加减法统一，不存在双零。

定义 2.1 (补码). n 位有符号整数 X 的补码值为：

$$T(X) = -x_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} x_i \cdot 2^i$$

范围： $[-2^{n-1}, 2^{n-1} - 1]$ （如 8 位为 $[-128, 127]$ ）。

示例 2.1 (补码转换). 8 位补码 1000 0001 的值:

$$-1 \cdot 2^7 + 1 \cdot 2^0 = -128 + 1 = -127$$

取反加一法 (快速转换): 原码 $\rightarrow$ 逐位取反 $\rightarrow$ 末位加一。

2.2.4 移码 (Biased Notation)

用于浮点数阶码。 $E_{\text{biased}} = E + \text{Bias}$, 单精度 $\text{Bias} = 127$, 双精度 $\text{Bias} = 1023$ 。

2.3 整数运算

2.3.1 加法器

$$\text{全加器: } S = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = AB + (A \oplus B)C_{\text{in}}$$

行波进位加法器 (Ripple Carry Adder): n 个全加器串联, 延迟 $O(n)$ 。超前进位加法器 (Carry Lookahead Adder): 并行计算进位, 延迟 $O(\log n)$ 。

2.3.2 溢出检测

- 同号相加得异号 $\rightarrow$ 溢出
- 补码: $A_{n-1} = B_{n-1} = 0$ 且 $S_{n-1} = 1$; 或 $A_{n-1} = B_{n-1} = 1$ 且 $S_{n-1} = 0$
- 硬件判断: $O = C_n \oplus C_{n-1}$

2.4 浮点数表示：IEEE 754

定义 2.2 (IEEE 754 单精度浮点数). 32 位: 1 位符号 S + 8 位移码阶码 E + 23 位尾数 M (隐含 1):

$$(-1)^S \times (1.M) \times 2^{E-127}$$

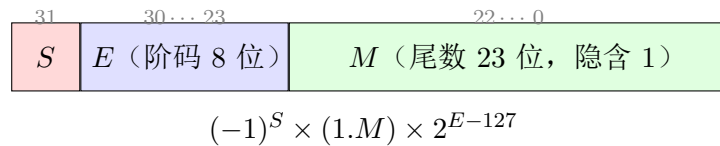


图 4: IEEE 754 单精度 (32 位) 位域划分

表 6: IEEE 754 格式参数

精度	总位数	阶码位	尾数位
单精度 (float)	32	8	23
双精度 (double)	64	11	52
扩展双精度	80	15	64

- 规格化数: $E \neq 0$ 且 $E \neq 255$, 尾数隐含 1
- 非规格化数 (denormal): $E = 0$, 尾数隐含 0, 用于表示极小数
- 无穷大: $E = 255$, $M = 0$
- NaN: $E = 255$, $M \neq 0$

舍入模式: 向偶数舍入 (Round to Nearest Even) (17)

2.5 浮点运算

加法: 对阶 $\rightarrow$ 尾数相加 $\rightarrow$ 规格化 $\rightarrow$ 舍入

乘法: 阶码相加 $\times$ 尾数相乘 $\rightarrow$ 规格化 $\rightarrow$ 舍入

(18)

2.6 字符编码

- **ASCII:** 7 位 (0-127), 扩展 ASCII 为 8 位
- **Unicode:** 统一的全球字符集
- **UTF-8:** 变长编码, ASCII 兼容 (1-4 字节), 互联网标准
- **GBK / GB 2312:** 中文编码标准

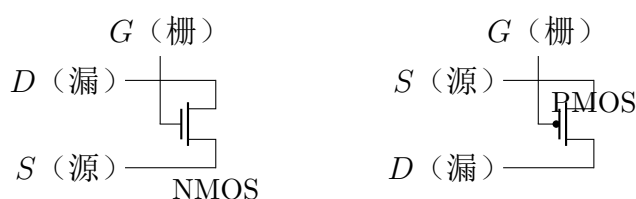
3 数字逻辑基础

3.1 CMOS 元件与门电路内部结构

现代数字芯片几乎全部用 CMOS（互补金属氧化物半导体）工艺实现。理解门电路的内部必须先认识晶体管及其开关模型。

3.1.1 MOSFET 开关模型

MOSFET（金属-氧化物-半导体场效应晶体管）可抽象为电压控制的开关，按导电载流子分两类：

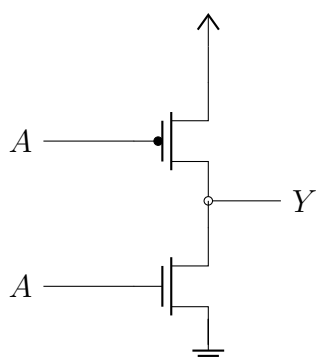


NMOS	PMOS
栅极 $G = 1$ （高电平）时导通	栅极 $G = 0$ （低电平）时导通
栅极 $G = 0$ 时断开	栅极 $G = 1$ 时断开
下拉：把输出拉到 GND（0）	上拉：把输出拉到 V_{DD} （1）

PMOS 与 NMOS 恰好互补，这是“CMOS”中 C（Complementary）的含义。

3.1.2 CMOS 反相器（NOT）

反相器由 1 个 PMOS（上拉）+ 1 个 NMOS（下拉）串联构成，栅极并联作为输入 A ，中间节点作为输出 Y ：

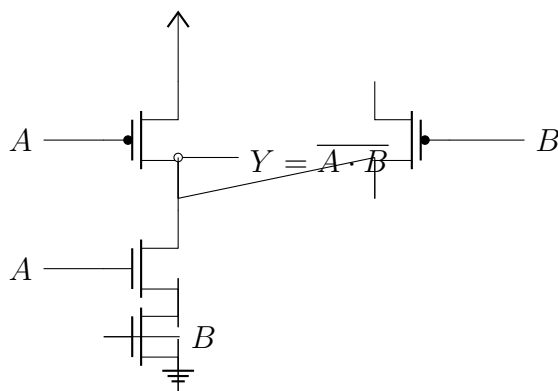


- $A = 1$ ：NMOS 导通（下拉），PMOS 断开 $\rightarrow Y = 0$
- $A = 0$ ：PMOS 导通（上拉），NMOS 断开 $\rightarrow Y = 1$

两个晶体管不同时导通，故静态功耗几乎为零——这是 CMOS 的核心优点。任何时刻输出要么被上拉要么被下拉，不存在浮空。

3.1.3 CMOS NAND（与非门）

NAND 由 2 个 PMOS 并联（上拉网络）+ 2 个 NMOS 串联（下拉网络）构成：



- $A = 1, B = 1$: 两个 NMOS 都导通，输出被下拉到 0
- 其余情况：至少一个 PMOS 导通，输出被上拉到 1

CMOS NOR 与之对偶：PMOS 串联、NMOS 并联。

3.1.4 由 NAND 派生其它门

NAND 是万能门，因为它本身就是 CMOS 的直接、最简实现；其它门用 NAND 组合即可：

- NOT: $Y = \overline{A \cdot A} = \overline{A}$ （两输入短接）
- AND: $Y = \overline{\overline{A \cdot B}}$ （NAND + NOT，共 6 个晶体管）
- OR: $Y = \overline{\overline{A} \cdot \overline{B}}$ （德·摩根）
- XOR: $Y = \overline{A} \cdot B + A \cdot \overline{B}$ （约需 4 个 NAND）

这就是”为什么 CMOS 工艺里 NAND 便宜、AND 反而要额外一层反相器”的原因：AND 是用 NAND 堆出来的。

3.2 布尔代数与门电路

数字电路的基础是布尔代数，变量取值 $\{0, 1\}$ ，三种基本运算：

$$\begin{aligned}
 &\text{与 (AND): } A \cdot B = 1 \iff A = 1 \wedge B = 1 \\
 &\text{或 (OR): } A + B = 1 \iff A = 1 \vee B = 1 \\
 &\text{非 (NOT): } \bar{A} = 1 \iff A = 0
 \end{aligned} \tag{19}$$

由这三种基本运算可组合出所有逻辑函数。下表给出常用门电路的真值表：

表 7: 基本门电路真值表

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

门电路在电路图中的标准符号如下：

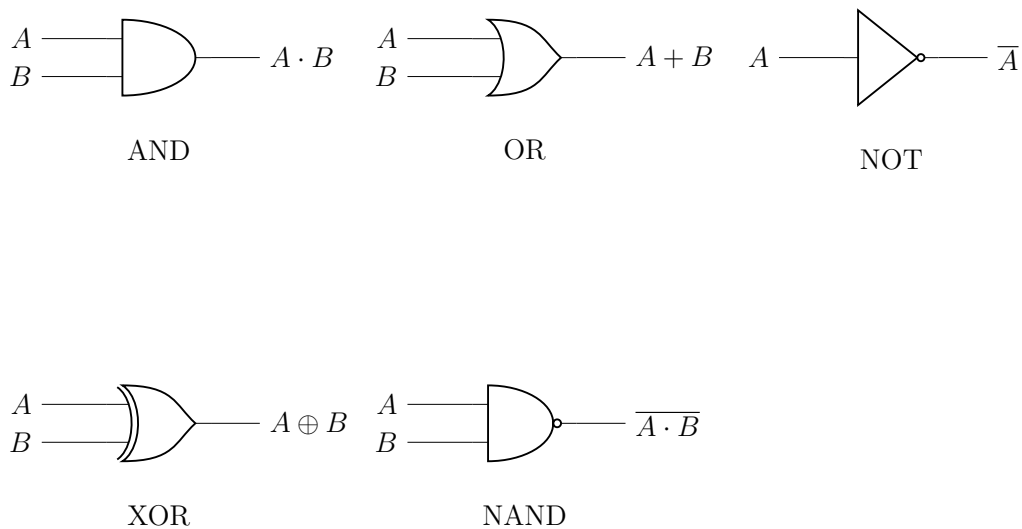


图 5: 基本门电路符号

NAND（与非门）是”万能门”——仅用 NAND 可实现 AND、OR、NOT 等全部逻辑，是数字集成电路最常见的实现方式。

3.3 逻辑表达式化简

3.3.1 代数化简

结合律、分配律、吸收律、德·摩根定律：

$$\overline{A \cdot B} = \overline{A} + \overline{B}, \quad \overline{A + B} = \overline{A} \cdot \overline{B} \quad (20)$$

德·摩根定律的电路等价性：一个 NAND 门等效于反相输入后的 OR 门；一个 NOR 门等效于反相输入后的 AND 门。这为 CMOS 实现提供了理论基础。

3.3.2 卡诺图 (Karnaugh Map)

最多 4-6 变量的图形化化简工具：将真值表画在网格上，圈出相邻的 1 形成最小项合并。

3.4 组合逻辑电路：加法器

加法器是从门电路到 CPU 运算器的桥梁。先从半加器开始。

3.4.1 半加器 (Half Adder)

半加器将两个比特相加，输出和位 (Sum) 与进位 (Carry)：

$$S = A \oplus B, \quad C_{\text{out}} = A \cdot B \quad (21)$$

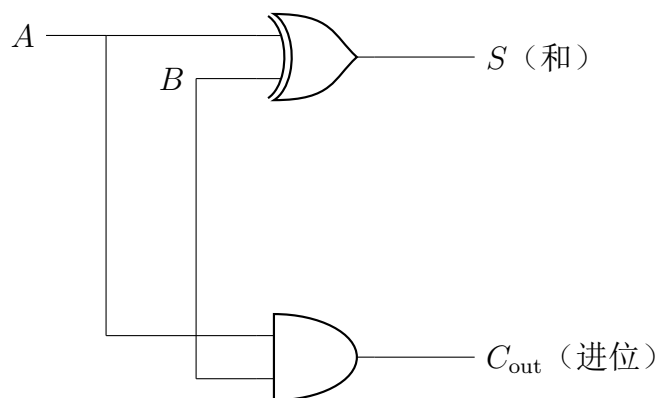


图 6: 半加器：一个 XOR 和一个 AND

3.4.2 全加器 (Full Adder)

全加器加上了来自低位的进位 C_{in} :

$$\begin{aligned} S &= A \oplus B \oplus C_{in} \\ C_{out} &= AB + (A \oplus B)C_{in} \end{aligned} \quad (22)$$

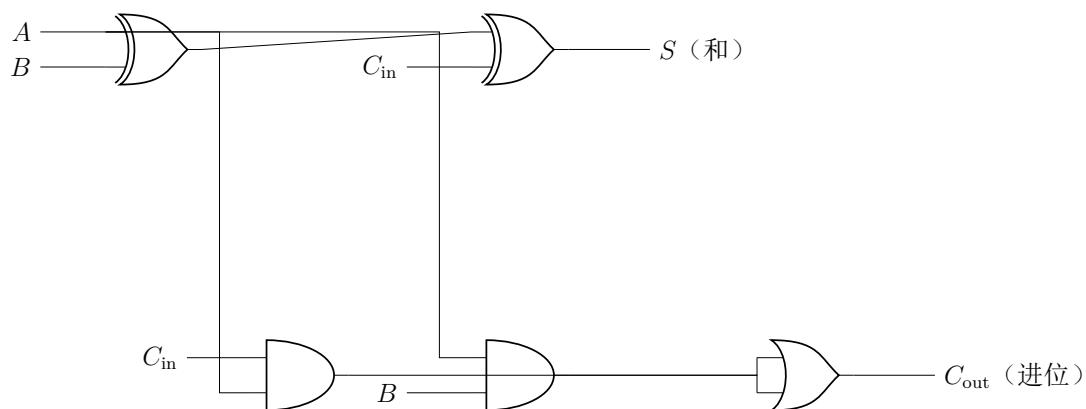


图 7: 全加器: 2 个半加器 + 1 个 OR 门

3.4.3 行波进位加法器 (Ripple Carry Adder)

将 n 个全加器串联，进位像”波浪”一样从低位向高位传播:

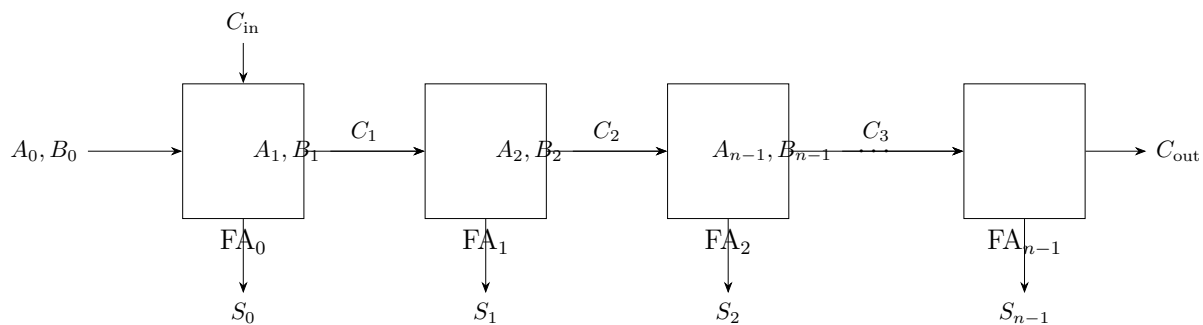


图 8: n 位行波进位加法器: 进位逐级传播, 延迟 $O(n)$

行波进位加法器的延迟为 $O(n)$ ——最高位必须等待所有低位进位算出。下一节介绍加速方案。

3.4.4 超前进位加法器 (Carry Lookahead Adder)

定义传播信号 $P_i = A_i \oplus B_i$ 和生成信号 $G_i = A_i \cdot B_i$ ，则进位的递推关系可展开为超前进位逻辑：

$$\begin{aligned} C_1 &= G_0 + P_0 C_0 \\ C_2 &= G_1 + P_1 G_0 + P_1 P_0 C_0 \\ C_3 &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \\ C_4 &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 \end{aligned} \quad (23)$$

每级进位由独立的”超前进位单元”(Carry Lookahead Unit, CLU) 并行产生，延迟 $O(\log n)$ 。实际设计中常用 4 位块 CLA 级联 (Ripple between blocks)。

3.4.5 溢出检测

- 同号相加得异号 $\rightarrow$ 溢出
- $O = C_n \oplus C_{n-1}$ ：最高位和次高位的进位不同即溢出

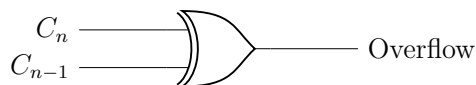


图 9: 溢出检测电路

3.5 组合逻辑电路：数据选择器与译码器

3.5.1 多路选择器 (Multiplexer, MUX)

2^n 路输入 + n 位选择信号 $\rightarrow$ 1 路输出。下面是 2-to-1 MUX 的门级实现：

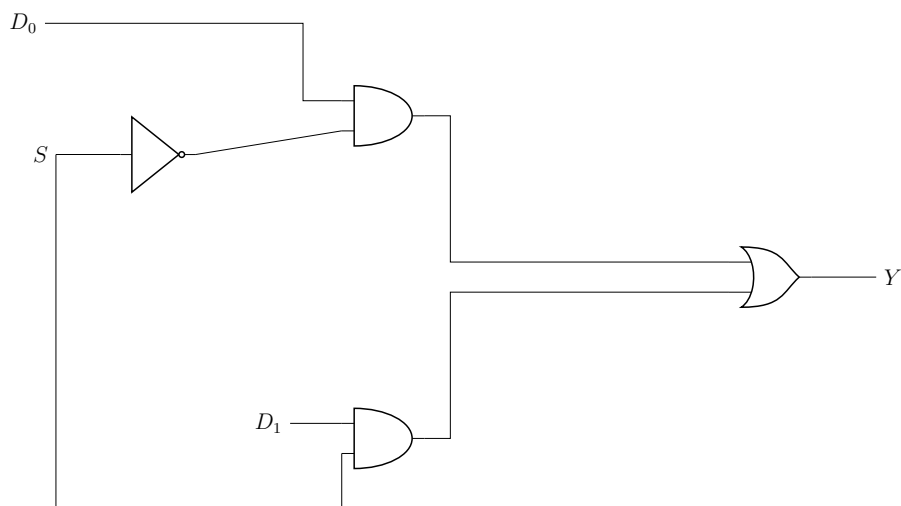


图 10: 2-to-1 MUX 门级实现: $Y = \bar{S} \cdot D_0 + S \cdot D_1$

$$Y = \bar{S} \cdot D_0 + S \cdot D_1 \quad (24)$$

抽象层面上, MUX 常用梯形符号表示 (宽边为输入, 窄边为输出, 底部为选择信号):

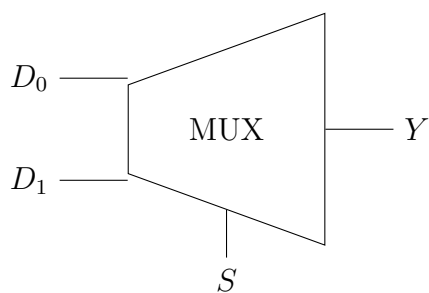


图 11: MUX 抽象符号

3.5.2 译码器 (Decoder)

n 位输入 $\rightarrow 2^n$ 位输出, 任意时刻仅有 1 位有效。2-to-4 译码器示意 (4 条垂直“总线”分别携带 $A_1, \overline{A_1}, A_0, \overline{A_0}$, 各 AND 门横向接入避免走线交叉):

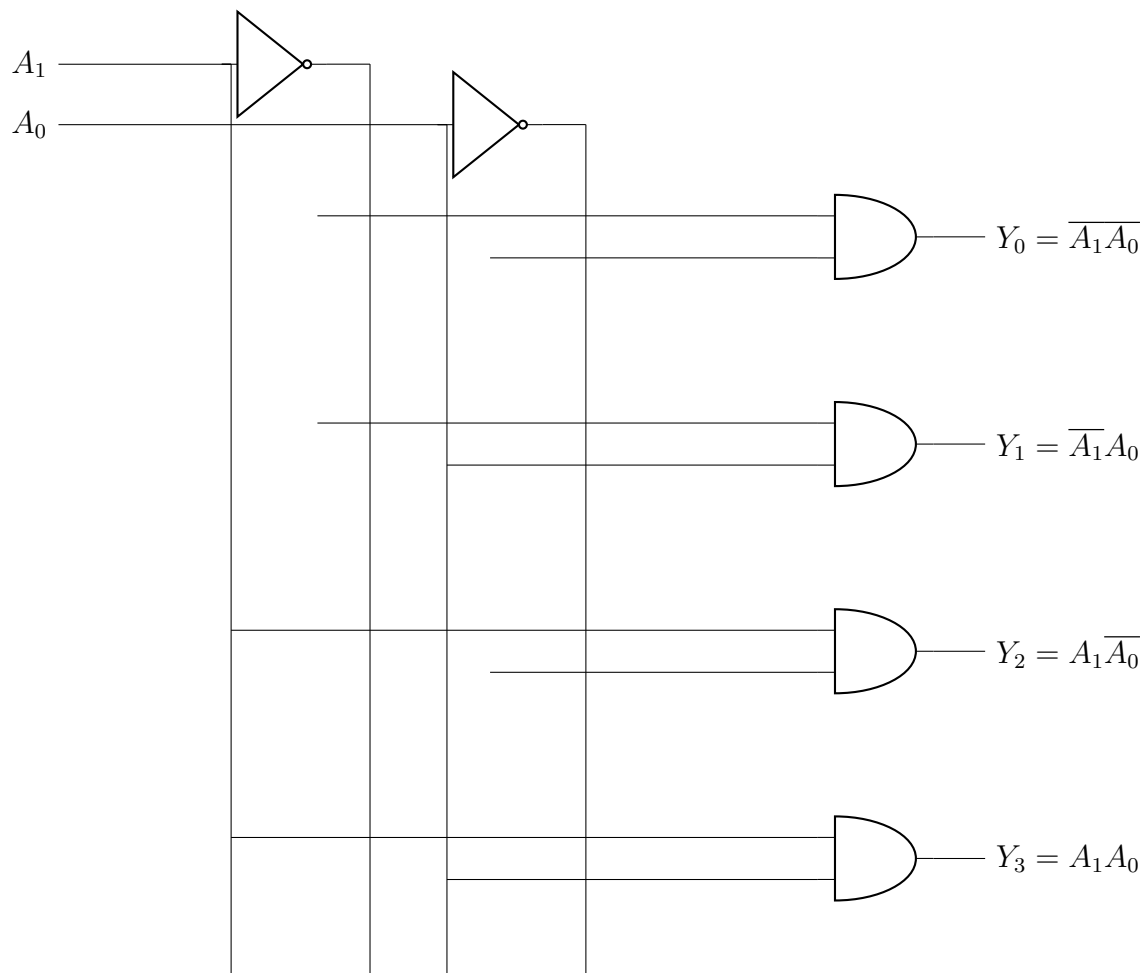


图 12: 2-to-4 译码器

3.6 算术逻辑单元 (ALU)

ALU 是 CPU 的核心运算部件。下图展示一个支持 AND、OR、ADD 三种运算的 1-bit ALU:

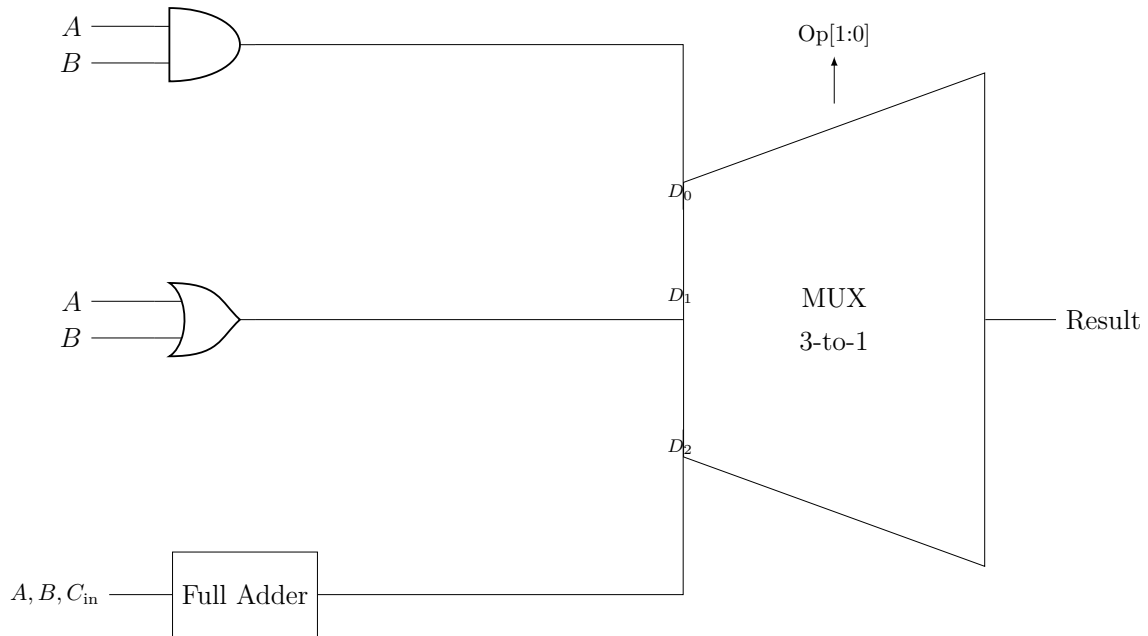


图 13: 1-bit ALU 结构

$$\text{Result} = \begin{cases} A \wedge B, & \text{Op} = 00 \\ A \vee B, & \text{Op} = 01 \\ A + B + C_{\text{in}}, & \text{Op} = 10 \end{cases} \quad (25)$$

32 位 ALU 由 32 个 1-bit ALU 并行构成, 进位在各位之间传递 (行波或超前进位)。

3.7 时序逻辑电路

时序逻辑电路的输出取决于当前输入 和历史状态（有记忆）。

3.7.1 SR 锁存器

SR 锁存器是最基本的存储单元，用两个 NOR 门交叉耦合构成：

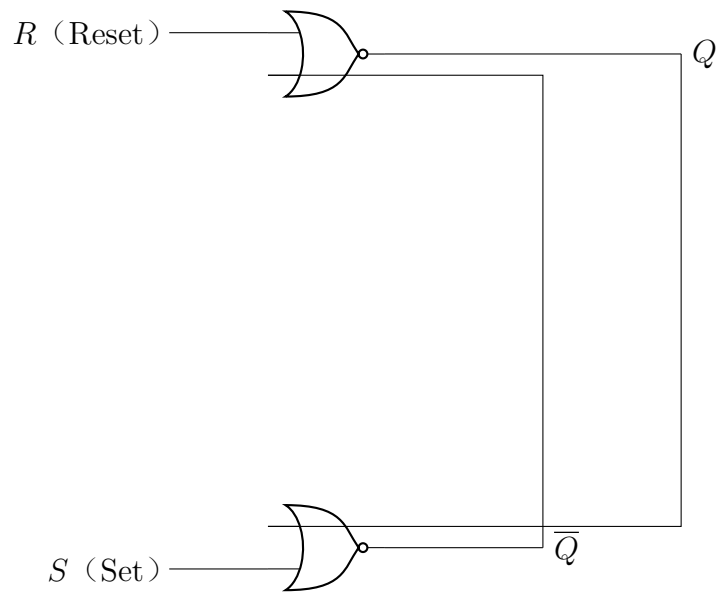


图 14: SR 锁存器（NOR 实现）

- $S = 0, R = 0$: 保持原值
- $S = 1, R = 0$: 置位 ($Q = 1$)
- $S = 0, R = 1$: 复位 ($Q = 0$)
- $S = 1, R = 1$: 非法 ($Q = \overline{Q} = 0$, 不稳定)

3.7.2 D 触发器 (D Flip-Flop)

SR 锁存器不方便定时，D 触发器是同步的 1-bit 存储单元，用时钟边沿（三角形符号）采样：

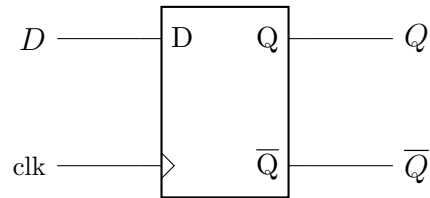


图 15: D 触发器

$$Q_{\text{next}} = D \quad (\text{时钟上升沿采样}) \quad (26)$$

D 触发器在时钟边沿采样 D 并更新 Q ，其余时间 Q 不变。 n 个 D 触发器并联即构成 n 位寄存器。

3.7.3 n 位寄存器

n 个 D 触发器共享时钟信号，同时输入/输出 n 位数据：

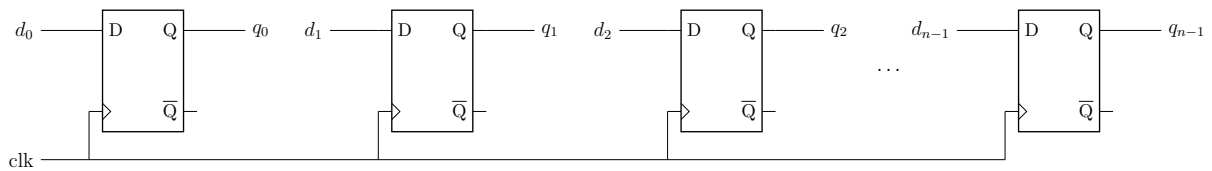


图 16: n 位寄存器: n 个 D 触发器共享时钟

3.8 有限状态机 (FSM)

定义 3.1 (FSM). 一个时序逻辑系统可抽象为有限状态机:

$$M = (S, I, O, \delta, \lambda, s_0) \quad (27)$$

S : 有限状态集, I : 输入集, O : 输出集, δ : 状态转移函数, λ : 输出函数, s_0 : 初始状态。

- **Moore 状态机**: 输出仅取决于当前状态
- **Mealy 状态机**: 输出取决于当前状态和输入

控制器就是 FSM 的经典应用: 状态是各执行阶段 (取指、译码、执行、访存、写回), 输入是指令操作码和条件标志, 输出是各数据通路组件的控制信号。

4 指令系统与汇编

4.1 指令系统：软件与硬件的接口

计算机系统可以粗略地划分为**软件层**与**硬件层**：软件层用高级语言、汇编语言描述算法，硬件层由晶体管、门电路与数据通路完成实际运算。二者必须约定一套共同语言，才能保证同一段程序在任意合规的实现上得到相同结果。这套共同语言就是**指令系统**，其完整的规范描述称为**指令集架构**（Instruction Set Architecture, ISA）。

定义 4.1 (指令集架构). 指令集架构是程序员可见的机器抽象：它规定了机器能够执行的全部指令、可以访问的寄存器与存储空间、支持的数据类型，以及指令的编码与寻址规则；但它不规定这些功能如何用硬件实现。*ISA* 是软件与硬件之间的一份**契约**。

4.1.1 ISA 在分层体系中的位置

同一个 ISA 可以有多种不同的硬件实现（例如 MIPS 的单周期、多周期、流水线实现，或 x86 的历代处理器）。只要它们对同一程序产生相同的结果，就都“实现了”该 ISA。把“做什么”（ISA）与“怎么做”（微架构）分离，带来的好处是：编译器只需面向 ISA 生成代码，硬件工程师只需按 ISA 设计电路，软硬件得以**独立演进**。

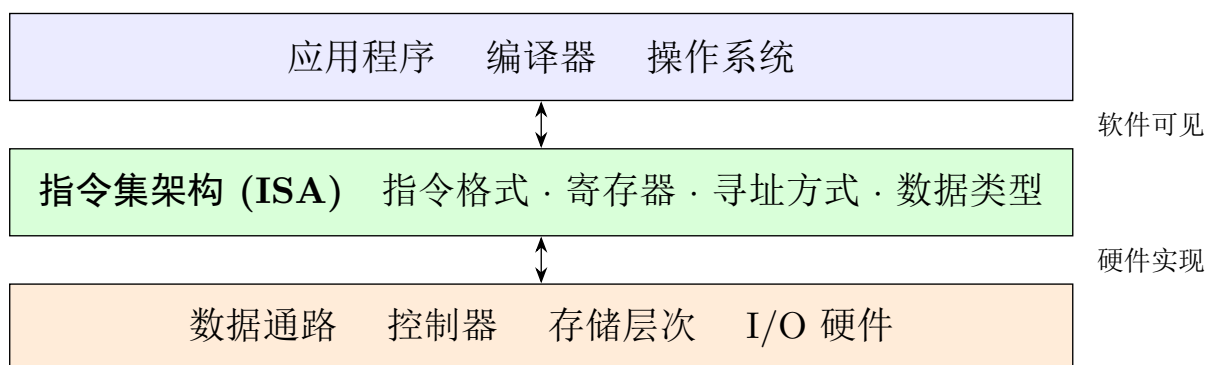


图 17: ISA 是软件与硬件之间的接口：上承软件、下启硬件

4.1.2 ISA 规定的内容

一套完备的 ISA 通常需要定义以下要素：

- **指令格式**：操作码与操作数如何编码，指令有多长；
- **寄存器组**：有哪些寄存器、数量、位宽与用途约定；
- **寻址方式**：如何由指令字段定位到操作数或内存地址；
- **数据类型**：支持哪些整数宽度、是否有浮点、字节序；

- **中断与异常：**发生异常时寄存器与栈如何维护；
- **I/O 模型：**使用内存映射 I/O 还是独立 I/O 端口。

本章以经典的 RISC 架构 **MIPS** 为主线讲解，并在末尾对比 CISC 与 RISC 的设计哲学。

4.2 指令格式

4.2.1 指令的组成

一条机器指令本质上是一串定长或变长的二进制位，通常由两部分组成：

- **操作码 (opcode)：**指明这条指令“做什么”；
- **操作数域：**指明“对谁做”，可以存放寄存器号、立即数、或地址偏移。

MIPS 采用**定长 32 位**指令，按用途分为三种格式：**R 型**（寄存器型）、**I 型**（立即数型）、**J 型**（跳转型）。

4.2.2 三种指令格式的字段划分

表 8: MIPS 三种指令格式的字段与位宽

格式	字段顺序（位宽）	总计	主要用途
R 型	op(6) + rs(5) + rt(5) + rd(5) + shamt(5) + funct(6)	32	算术、逻辑、移位
I 型	op(6) + rs(5) + rt(5) + immediate(16)	32	立即数运算、访存、分支
J 型	op(6) + address(26)	32	无条件跳转

表 9: MIPS 各字段的含义

字段	位宽	含义
op	6	操作码，最高 6 位；全 0 表示 R 型，由 funct 决定具体运算
rs	5	第一源操作数寄存器号（0 ~ 31）
rt	5	第二源操作数寄存器号，或 I 型的目标寄存器号
rd	5	R 型的目标寄存器号
shamt	5	移位量（shift amount），仅移位指令使用
funct	6	功能码，与 op=0 配合区分 R 型运算
immediate	16	立即数或地址偏移，参与符号扩展
address	26	跳转目标地址的低 26 位（字地址）

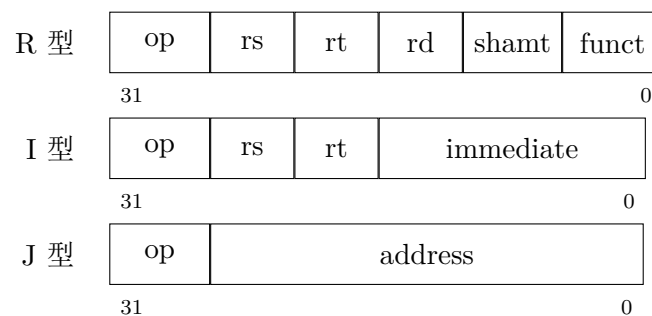


图 18: MIPS 三种指令格式的位域划分（R: 6/5/5/5/5/6, I: 6/5/5/16, J: 6/26）

位域从左到右对应高位到低位，最高位 MSB 为第 31 位、最低位 LSB 为第 0 位。这样划分的共同点是：**op 固定占最高 6 位**，因此译码器可以先用这 6 位快速判断格式

类别，再决定如何解释剩余字段。这种“先看前缀、再分派”的思路正是流水线译码的关键。

4.3 寻址方式

指令要访问操作数，就必须通过某种**寻址方式**计算出操作数所在的位置。若操作数在存储器中，则计算出的位置称为**有效地址**（Effective Address, EA）。下面逐一介绍九种经典寻址方式。

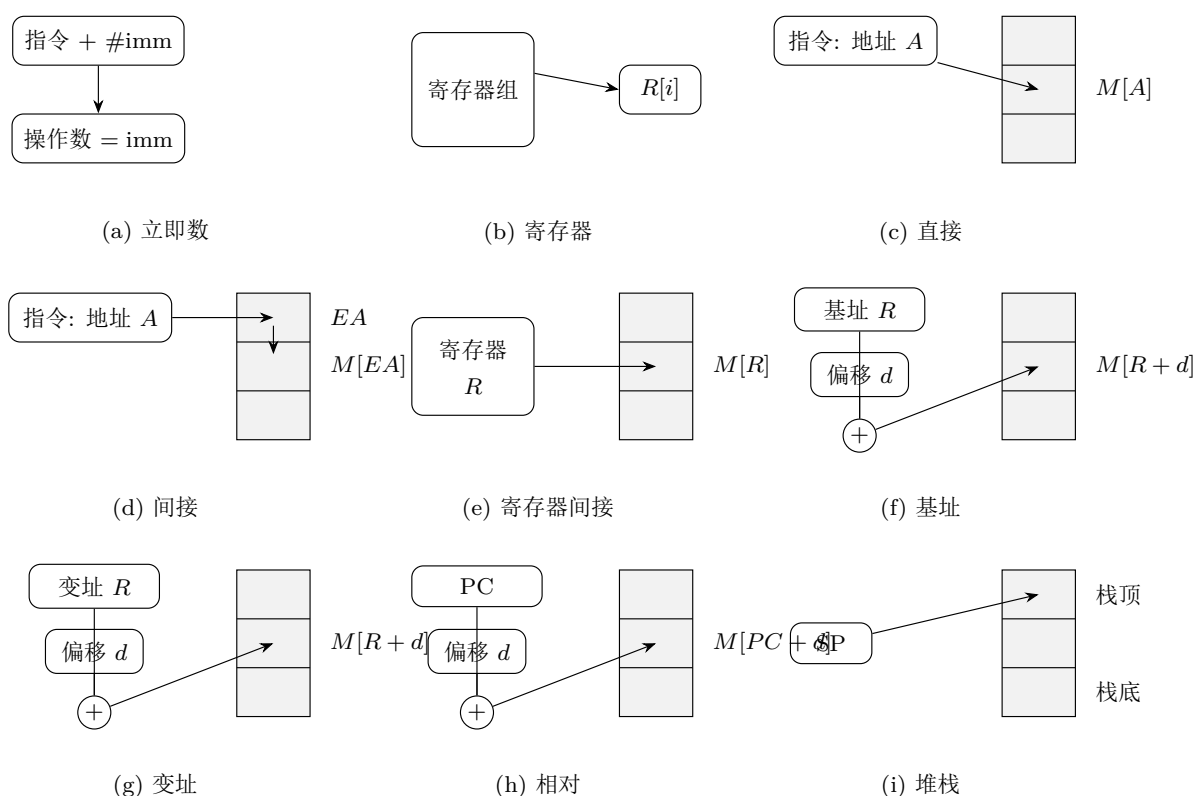


图 19: 九种寻址方式示意（方框为寄存器/指令，右侧分格为存储器）

4.3.1 各种寻址方式的有效地址计算式

- **立即数寻址**：操作数就在指令中，不存在访存，操作数 = imm；
- **寄存器寻址**：操作数 = $R[i]$ ，速度最快；
- **直接寻址**： $EA = A$ ，指令直接给出内存地址；
- **间接寻址**： $EA = M[A]$ ，内存中的内容再作为地址，可一次访存两次取值；
- **寄存器间接寻址**： $EA = R[i]$ ，寄存器里存的是地址；

- 基址寻址: $EA = R_{\text{base}} + d$, 适合定位数组/结构体基址;
- 变址寻址: $EA = R_{\text{index}} + d$, 适合循环遍历数组;
- 相对寻址: $EA = PC_{\text{next}} + (d \ll 2)$, 用于分支, 代码可重定位;
- 堆栈寻址: $EA = SP$, 按下推 (push, $SP \leftarrow SP - 4$) 与弹出 (pop, $SP \leftarrow SP + 4$) 存取。

表 10: 寻址方式对比

方式	操作数位置	有效地址 EA	访存次数	MIPS 支持
立即数	指令中	无	0	addi
寄存器	寄存器	无	0	add
直接	存储器	A	1	少见
间接	存储器	$M[A]$	2	无
寄存器间接	存储器	$R[i]$	1	基址的特例
基址	存储器	$R_{\text{base}} + d$	1	lw 8(\$t1)
变址	存储器	$R_{\text{index}} + d$	1	借用基址
相对	存储器	$PC + d$	1	beq / bne
堆栈	存储器	SP	1	push / pop

4.4 MIPS 常用指令分类

MIPS 指令按功能大致分为五类, 表 11 给出各类代表指令及其格式。

表 11: MIPS 指令分类与示例

类别	代表指令	格式	功能
算术运算	add sub addi mult	R / I	加减乘除, 注意溢出
逻辑运算	and or xor sll srl	R / I	按位运算与移位
访存	lw sw lb sb	I	Load/Store, 只有它们访存
分支	beq bne slt	I / R	比较后条件跳转
跳转	j jal jr	J / R	无条件跳转、过程调用

4.4.1 示例机器码

表 12: 常用 MIPS 指令及其 32 位机器码

汇编指令	格式	语义	机器码
<code>add \$t0,\$t1,\$t2</code>	R	$\$t0 = \$t1 + \$t2$	0x012A4020
<code>addi \$t0,\$t1,100</code>	I	$\$t0 = \$t1 + 100$	0x20940064
<code>lw \$t0,8(\$t1)</code>	I	$\$t0 = M[\$t1+8]$	0x8D280008
<code>sw \$t0,8(\$t1)</code>	I	$M[\$t1+8] = \$t0$	0xAD280008
<code>beq \$t0,\$t1,4</code>	I	if $\$t0=\$t1$ goto $PC+4+16$	0x11090004
<code>j 0x00400000</code>	J	无条件跳到 0x00400000	0x08100000

4.4.2 完整例题：把汇编翻译成机器码

示例 4.1 (汇编 $\rightarrow$ 机器码). 把 `add $t0, $t1, $t2` 翻译为 32 位机器码。已知 R 型指令：`add` 的 $op = 0$ 、 $funct = 0x20$ ；寄存器号 $\$t0=8$, $\$t1=9$, $\$t2=10$ 。

1. 确定格式：`add` 是 R 型，目标寄存器写入 rd ，源在 rs 与 rt 。因此 $rd = \$t0 = 8$, $rs = \$t1 = 9$, $rt = \$t2 = 10$ 。
2. 逐字段转二进制：

$$\underbrace{000000}_{op} \underbrace{01001}_{rs=9} \underbrace{01010}_{rt=10} \underbrace{01000}_{rd=8} \underbrace{00000}_{shamt} \underbrace{100000}_{funct=0x20}$$

3. 拼接为 32 位并按每 4 位一组转十六进制：

$$0000\ 0001\ 0010\ 1010\ 0100\ 0000\ 0010\ 0000 = 0x012A4020$$

故机器码为 `0x012A4020`，与表 12 一致。

示例 4.2 (机器码 $\rightarrow$ 汇编：反向译码). 给定机器码 `0x012A4020`，还原其汇编指令。

1. 展开为二进制：`0000 0001 0010 1010 0100 0000 0010 0000`。
2. 按 R 型切分字段： $op = 000000$ (即 R 型)， $rs = 01001 = 9$ ， $rt = 01010 = 10$ ， $rd = 01000 = 8$ ， $shamt = 00000$ ， $funct = 100000 = 0x20$ 。
3. 查功能码表： $op = 0$ 且 $funct = 0x20$ 表示 `add`。
4. 组装： $rd = 8$ ($\$t0$)， $rs = 9$ ($\$t1$)， $rt = 10$ ($\$t2$)。

结果为 `add $t0, $t1, $t2`。整个译码过程可归结为“取指 $\rightarrow$ 定格式 $\rightarrow$ 切字段 $\rightarrow$ 查表 $\rightarrow$ 组装”五步。

4.5 CISC 与 RISC

按照指令系统的复杂度，处理器可分为两大流派：**CISC**（复杂指令集计算机）与**RISC**（精简指令集计算机）。

表 13: CISC 与 RISC 对比

特征	RISC (MIPS、ARM、RISC-V)	CISC (x86、VAX)
指令长度	定长（如 32 位）	变长（1~15 字节）
指令数量	少（约 100 条）	多（约 300 条以上）
寻址方式	简单，仅 Load/Store 访存	复杂，运算可直接访存
寄存器数量	多（32 个通用寄存器）	少（8 个，x86-64 扩展到 16）
指令执行周期	大多单周期，利于流水	周期不一，需微程序
控制器	硬布线为主	微程序为主
编译负担	重，依赖编译器优化	轻，硬件承担更多
设计哲学	让常用指令更快	让指令更贴近高级语言

RISC 的核心理念是：**指令简单规整** → **便于流水线** → **提高主频与吞吐**。由于访存只有 Load/Store 两条指令，运算指令的延迟可控，流水线设计大为简化。这也是现代 RISC（ARM、RISC-V）在移动与嵌入式领域占据主导的原因；而 x86 通过把复杂指令在内部译码为类 RISC 的微操作，兼顾了兼容性与性能。

4.6 过程调用与栈帧

4.6.1 调用约定

过程（函数）调用需要解决四个问题：如何传递参数、如何保存返回地址、如何保存调用者/被调用者的寄存器、如何返回结果。MIPS 用一组寄存器约定来规范这些行为，见表 14。

表 14: MIPS 寄存器用途约定

寄存器号	名称	用途
\$0	\$zero	恒为 0
\$1	\$at	汇编器保留
\$2-\$3	\$v0 \$v1	函数返回值
\$4-\$7	\$a0-\$a3	前 4 个函数参数
\$8-\$15	\$t0-\$t7	临时变量（caller-saved）
\$16-\$23	\$s0-\$s7	保存变量（callee-saved，需恢复）
\$29	\$sp	栈指针
\$30	\$fp	帧指针
\$31	\$ra	返回地址

约定要点：

- **参数传递：**前 4 个参数放 \$a0-\$a3，超出部分通过栈传递；
- **返回值：**放在 \$v0（必要时 \$v1）；
- **返回地址：**jal 把下一条指令地址存入 \$ra，返回时用 jr \$ra；
- **寄存器保护：**\$t 系列由调用者负责保存（若需要），\$s 系列由被调用者保存并恢复。

4.6.2 栈帧结构

被调用过程若需要保存 $\$ra$ 、 $\$s$ 寄存器或使用局部变量，就在栈上分配一块**栈帧** (stack frame)。MIPS 的栈向**低地址**方向生长：分配时 SP 减小，释放时 SP 增大。典型布局如图 20。

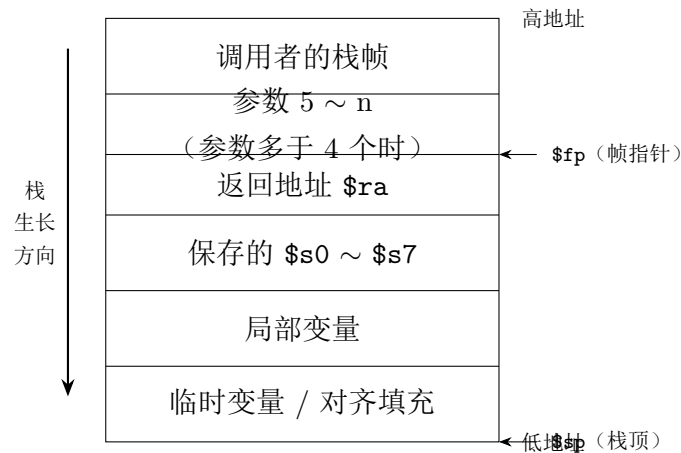


图 20: MIPS 栈帧结构：栈向低地址生长， $\$sp$ 指向当前帧顶

一个遵循约定的过程体可用如下汇编模板描述：

Listing 1: MIPS 过程调用模板：保存/恢复现场

```

1  # ---- 建立栈帧 ----
2  addi $sp, $sp, -12    # 分配 12 字节栈帧
3  sw    $ra, 8($sp)     # 保存返回地址
4  sw    $s0, 4($sp)     # 保存 callee-saved 寄存器
5  sw    $s1, 0($sp)
6  # ---- 过程体 ----
7  # ... 使用 $s0/$s1 与局部变量 ...
8  move $v0, $s0         # 设置返回值
9  # ---- 恢复现场 ----
10 lw    $s1, 0($sp)
11 lw    $s0, 4($sp)
12 lw    $ra, 8($sp)
13 addi $sp, $sp, 12     # 释放栈帧
14 jr    $ra             # 返回调用者

```

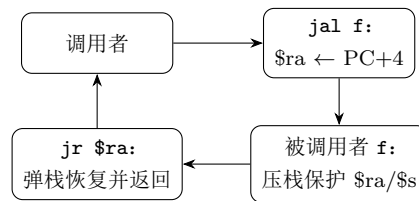


图 21: 过程调用与返回的控制流: jal 保存返回地址, jr \$ra 据此返回

4.6.3 示例: 数组求和

Listing 2: $a[0] + a[1] + \dots + a[9]$ 计算 $a[0] + a[1] + \dots + a[9]$

示例 4.3 (数组求和的汇编实现).

```

1  li    $t0, 0          # sum = 0
2  li    $t1, 0          # i = 0
3  la    $t2, array      # $t2 = &a[0]
4  loop:
5  lw    $t3, 0($t2)      # $t3 = a[i]      (基址偏移寻址)
6  addu  $t0, $t0, $t3    # sum += $t3
7  addi  $t1, $t1, 1      # i++
8  addi  $t2, $t2, 4      # $t2 += 4 (下一个字)
9  blt   $t1, 10, loop    # if i < 10 goto loop
10 done:
11  move  $v0, $t0        # 返回 sum
  
```

其中 *lw* 使用了基址偏移寻址 ($EA = \$t2 + 0$), *blt* 由 *slt+bne* 组合实现, 体现了 RISC “用简单指令组合出复杂语义” 的思想。

4.7 本章小结

- **ISA 是软硬件接口:** 它规定了指令格式、寄存器、寻址方式与数据类型, 是可移植性的基础; 同一 ISA 可有多种微架构实现。
- **MIPS 三种格式:** R 型 (6/5/5/5/5/6)、I 型 (6/5/5/16)、J 型 (6/26), *op* 固定占最高 6 位, 便于快速译码。
- **寻址方式统一用有效地址 *EA* 刻画:** 立即/寄存器无需访存, 直接/间接/寄存器间接/基址/变址/相对/堆栈各有公式与适用场景。
- **指令分类:** 算术、逻辑、访存、分支、跳转; 只有 Load/Store 访存是 RISC 的关键特征。
- **汇编与机器码可双向翻译:** 定格式 → 切字段 → 转进制 (或查表)。

- **过程调用依赖调用约定：**\$a 传参、\$v 返回、\$ra 存返回地址、\$t/\$s 划分保存责任，\$sp 维护向低地址生长的栈帧。

4.8 常见误区

- **把 ISA 与微架构混为一谈：**ISA 是软件可见的抽象，微架构是具体电路实现；同一 ISA 的性能可以差别巨大。
- **认为所有 MIPS 指令都能访存：**只有 lw/sw 等 Load/Store 指令访问内存，算术逻辑指令只操作寄存器。
- **混淆直接寻址与间接寻址：**直接寻址 $EA = A$ 只访存一次；间接寻址 $EA = M[A]$ 需访存两次。
- **混淆基址与变址：**两者公式都是 $R + d$ ，区别在语义——基址寄存器放“基址”、偏移变，变址寄存器放“下标”、基址固定（现代处理器常不区分）。
- **相对寻址忘记左移：**MIPS 分支偏移以“字”为单位， $EA = PC + (\text{offset} \ll 2)$ ，不是简单的 $PC + \text{offset}$ 。
- **忘记恢复 \$ra：**若过程内还会调用其它过程，必须先保存 \$ra 再调用，否则返回地址被覆盖，jr \$ra 将跳错。
- **栈指针方向弄反：**MIPS 栈向低地址生长，分配栈帧用 `addi $sp, $sp, -n`，释放在用 `+n`。
- **误解寄存器号位宽：**32 个寄存器只需 5 位 ($2^5 = 32$)，因此 rs/rt/rd 各占 5 位，而不是把 32 位都留给一个操作数。

5 处理器：数据通路

数据通路 (datapath) 是 CPU 中”搬运和处理数据”的部分：它由寄存器、存储器、运算器和连线组成，负责按指令的语义让数据在部件之间流动。控制器决定”怎么动”，数据通路决定”数据往哪走”。本章以经典 MIPS 单周期实现为蓝本，逐层拆解各部件的输入输出、五步执行流程、四类指令的数据流、控制信号的位置以及决定时钟周期的关键路径。

5.1 数据通路的组成部件

定义 5.1 (数据通路). 数据通路是执行部件 (*ALU*、寄存器堆、存储器) 与它们之间数据连线的集合。任一时刻，数据通路的行为由一组控制信号决定：多路选择器选通哪条支路、寄存器堆是否写入、存储器是读还是写。

单周期数据通路由六个核心部件构成。理解每一个部件的输入与输出是分析数据流的第一步。

表 15: 数据通路核心部件的输入与输出

部件	输入	输出
程序计数器 PC	下一条指令地址 (由 PC 来源 MUX 给出)、时钟	当前指令地址
指令存储器	指令地址	32 位指令字 (机器码)
寄存器堆 (32×32 位)	读地址 <i>rs</i> 、 <i>rt</i> ；写地址 (<i>rt</i> 或 <i>rd</i>)；写数据；RegWrite	读出数据 A、读出数据 B
ALU	操作数 A、操作数 B、ALUOp	运算结果、Zero (结果是否为零)
数据存储器	地址、写数据、MemRead、MemWrite	读出数据
符号扩展单元	16 位立即数	32 位符号扩展立即数

各部件要点：

- **PC**：唯一保存”程序位置”的时序部件。它只在时钟沿更新，新值来自前端的 PC 来源 MUX ($PC+4$ 或分支目标)。
- **指令存储器**：只读，输入地址、输出该地址处的 32 位指令字。取指与访存是两个独立的存储器 (指令存储器 vs 数据存储器)，这正是”哈佛结构”的雏形。
- **寄存器堆**：双读单写。 $\$0$ 恒为 0，写操作在时钟沿发生；单周期中”读旧值、写新值”发生在同一周期内，读口看到的是本周期的旧值。

- **ALU**：运算核心，既能做加减逻辑（R 型），也能做地址加法（lw/sw），还能做相等比较（beq）。
- **数据存储器**：由 MemRead/MemWrite 控制方向。读与写互斥，单周期内每个周期至多一次访存。
- **符号扩展单元**：把 16 位立即数按符号位扩展成 32 位。注意区分“零扩展”（用于无符号逻辑运算）与“符号扩展”（用于地址偏移与有符号立即数）。

5.2 单周期执行的五个阶段

无论指令类型如何，单周期 CPU 都在一个时钟周期内走完从取指到写回的全部动作。从逻辑上可分为五步：

表 16: 单周期执行的五个阶段

阶段	名称	主要动作与部件
IF	取指	PC 送指令存储器，取出 32 位指令；同时计算 PC+4
ID	译码/读寄存器	拆出 op/rs/rt/rd/imm；寄存器堆读出 rs、rt；符号扩展 imm
EX	执行/地址计算	ALU 运算：R 型算术逻辑、lw/sw 地址、beq 比较
MEM	访存	仅 lw 读数据存储器，sw 写数据存储器，其余指令空转
WB	写回	结果写回寄存器堆（R 型用 ALU 结果，lw 用存储器数据）

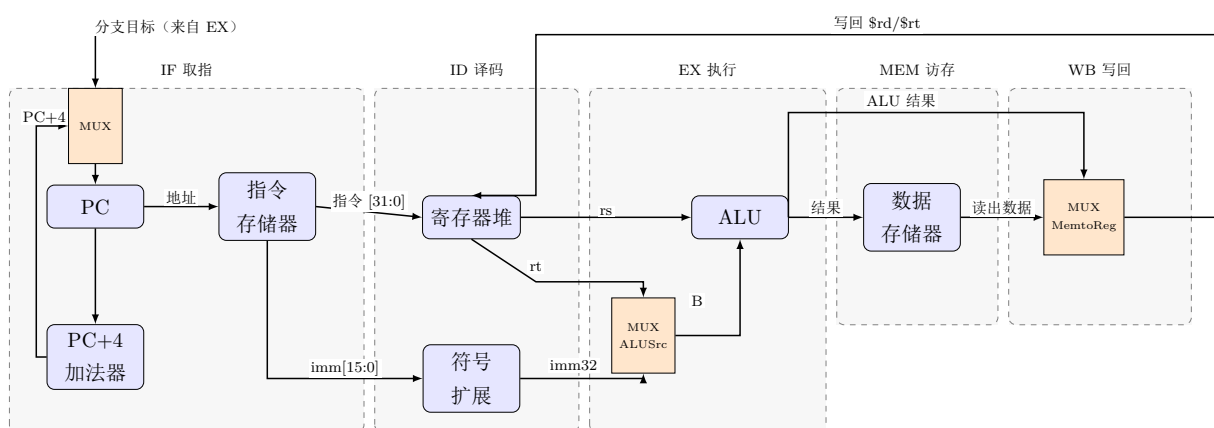


图 22: MIPS 单周期数据通路总图（虚线框为五个逻辑阶段）

图 22 中三个 MUX 是数据通路的“岔路口”：PC 来源 MUX (PC+4 / 分支目标)、ALU 第二输入 MUX (rt / imm32)、写回数据 MUX (ALU 结果 / 存储器数据)。控制信号正是通过控制这三个 MUX 以及寄存器堆和存储器的读写使能来完成指令分派。

5.3 四类指令的数据流

同一张数据通路之所以能执行不同指令，是因为控制信号为每条指令“点亮”不同的路径。下面用四张子图分别标出 R 型、lw、sw、beq 的活跃部件，灰色（浅蓝）方块为参与运算的部件。

5.3.1 R 型指令 (add/sub/and/or/slt)

$$\text{add } \$rd, \$rs, \$rt \quad \Longrightarrow \quad \$rd \leftarrow \$rs + \$rt \quad (28)$$

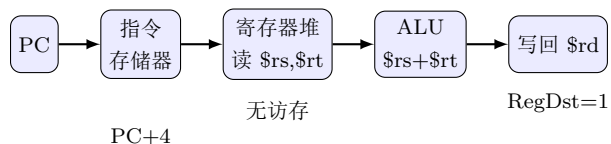


图 23: R 型指令数据流: 寄存器堆两读一写, ALU 直接产出, 不访存

R 型的特征: **RegDst=1** (写回目标取 rd), **ALUSrc=0** (ALU 第二输入来自寄存器 rt), **MemtoReg=0** (写回 ALU 结果), **RegWrite=1**, **MemRead=MemWrite=0**。

5.3.2 I 型: lw (取字)

$$\text{lw } \$rt, \text{offset}(\$rs) \implies \$rt \leftarrow M[\$rs + \text{SignExt}(\text{offset})] \quad (29)$$

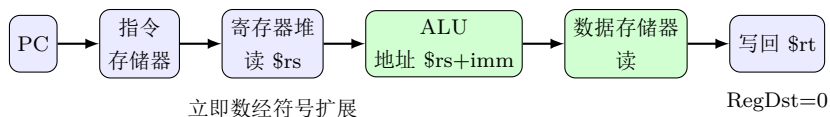


图 24: lw 数据流: ALU 算地址, 访存读出的数据写回 rt

lw 的特征: **ALUSrc=1** (ALU 第二输入取符号扩展立即数), **MemRead=1**, **MemtoReg=1** (写回的是存储器数据), **RegDst=0** (写回目标取 rt), **RegWrite=1**。它是全部指令中路径最长的一条, 第 5.5 节将说明其影响。

5.3.3 I 型：sw（存字）

$$\text{sw } \$rt, \text{offset}(\$rs) \implies M[\$rs + \text{SignExt}(\text{offset})] \leftarrow \$rt \quad (30)$$

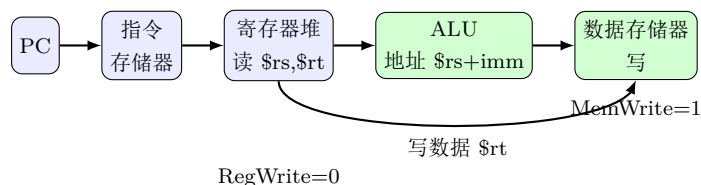


图 25: sw 数据流：与 lw 共享地址计算，但数据由寄存器堆流向存储器

sw 与 lw 的差异在于方向相反：地址仍由 ALU 计算，但 \$rt 的值作为写数据直接送入数据存储器；同时 **RegWrite=0**、**MemWrite=1**、**MemRead=0**，写回阶段实际不产生效果。这也是 sw 与 lw 共用同一条数据通路的直观体现。

5.3.4 I 型：beq（相等则分支）

$$\text{beq } \$rs, \$rt, \text{label} \implies \text{若 } \$rs = \$rt, \text{ 则 } PC \leftarrow PC + 4 + \text{offset} \times 4 \quad (31)$$

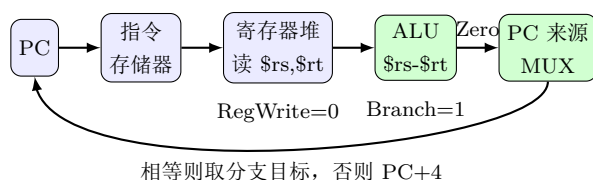


图 26: beq 数据流：ALU 相减产生 Zero，控制 PC 来源 MUX

beq 的特征：**ALUSrc=0**（两个操作数都来自寄存器，做 \$rs-\$rt），**RegWrite=0**、**MemRead=MemWrite=0**（不写回、不访存），**Branch=1**。分支目标由另一个加法器计算：先把 16 位 offset 符号扩展，再左移 2 位（×4，因为指令按字对齐），最后与 PC+4 相加。

5.4 控制信号在数据通路中的位置

控制器从指令中提取 `op` 与 `funct`，产生全部控制信号。它们并不集中在一处，而是分散地作用在各个部件上：

表 17: 各控制信号作用的部件

控制信号	作用部件	作用
RegDst	写回寄存器号 MUX	选 <code>rt</code> (0) 或 <code>rd</code> (1) 作为写地址
RegWrite	寄存器堆	写使能
ALUSrc	ALU 输入 MUX	第二操作数选 <code>rt</code> (0) 或立即数 (1)
MemRead / MemWrite	数据存储器	读/写使能
MemtoReg	写回数据 MUX	写回选 ALU 结果 (0) 或存储器数据 (1)
Branch / PCSrc	PC 来源 MUX	选 <code>PC+4</code> 或分支目标
ALUOp	ALU 控制逻辑	00 加、01 减、10 由 <code>funct</code> 决定

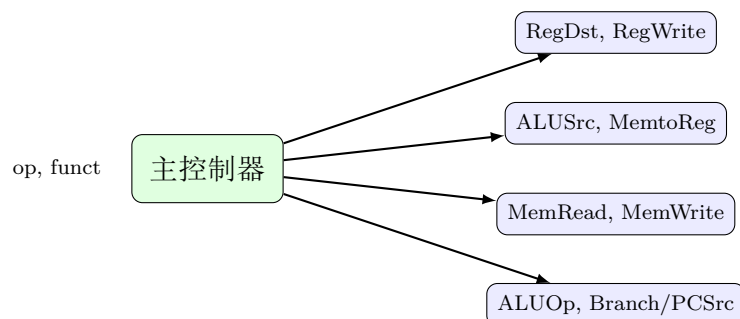


图 27: 主控制器把控制信号分发到各数据通路部件

ALU 输入来源的选择：ALU 的第一操作数固定来自寄存器堆的 A 口 (`rs`)；第二操作数经 `ALUSrc` MUX 在“寄存器 B 口 (`rt`)”与“符号扩展立即数”之间二选一。R 型与 `beq` 选 `rt`，`lw/sw` 选立即数。

写回来源的选择：写回数据经 `MemtoReg` MUX 在“ALU 结果”与“存储器读出数据”之间二选一。R 型选 ALU 结果，`lw` 选存储器数据；写地址则由 `RegDst` 在 `rt` 与 `rd` 间选择。四条指令的控制信号取值汇总如下。

表 18: 四类指令的控制信号取值

信号	R 型	lw	sw	beq
RegDst	1	0	×	×
RegWrite	1	1	0	0
ALUSrc	0	1	1	0
MemRead	0	1	0	0
MemWrite	0	0	1	0
MemtoReg	0	1	×	×
Branch	0	0	0	1
ALUOp	10	00	00	01

表中 × 表示“无关项”（don't care）：当 RegWrite=0 时，写地址与写回数据都不会被使用，因此 RegDst 与 MemtoReg 取任意值都不影响结果。

5.5 关键路径与性能

数据通路中每条指令走过的最长组合逻辑延迟称为该指令的**关键路径**（critical path）。单周期 CPU 的时钟周期必须容纳所有指令中最长的那条，否则快路径上早已算完的结果也会被最慢的指令拖到下一个时钟沿。

对 MIPS 单周期实现，lw 的关键路径最长，它依次穿过：

PC（时钟到输出）→ 指令存储器（取指）→ 寄存器堆读 → ALU（算地址）→ 数据存储器（读）→ 寄存器堆写（建立）。

其它指令短一截：R 型不含访存，sw 不含写回，beq 既不访存也不写回。

表 19: 各部件延迟假设（用于关键路径计算）

部件	延迟 (ps)
PC 时钟到输出 t_{pcq}	30
指令存储器访问 t_{im}	250
寄存器堆读 t_{read}	150
ALU 运算 t_{alu}	200
数据存储器访问 t_{mem}	250
寄存器堆写建立 t_{setup}	100

示例 5.1 (单周期时钟周期的计算). 给定表 19 的延迟, 求四条指令各自的关键路径延迟, 进而确定时钟周期与主频。

R 型 ($PC \rightarrow$ 取指 $\rightarrow$ 读寄存器 $\rightarrow ALU \rightarrow$ 写回):

$$t_{pcq} + t_{im} + t_{read} + t_{alu} + t_{setup} = 30 + 250 + 150 + 200 + 100 = 730 \text{ ps}. \quad (32)$$

lw (最长):

$$t_{pcq} + t_{im} + t_{read} + t_{alu} + t_{mem} + t_{setup} = 30 + 250 + 150 + 200 + 250 + 100 = 980 \text{ ps}. \quad (33)$$

sw (无写回):

$$t_{pcq} + t_{im} + t_{read} + t_{alu} + t_{mem} = 30 + 250 + 150 + 200 + 250 = 880 \text{ ps}. \quad (34)$$

beq (无访存、无写回):

$$t_{pcq} + t_{im} + t_{read} + t_{alu} = 30 + 250 + 150 + 200 = 630 \text{ ps}. \quad (35)$$

最长路径为 *lw* 的 980 ps, 故时钟周期为

$$T_{clk} = \max = 980 \text{ ps}, \quad f = \frac{1}{T_{clk}} = \frac{1}{980 \times 10^{-12} \text{ s}} \approx 1.02 \text{ GHz}. \quad (36)$$

由于单周期 $CPI = 1$, 平均每条指令耗时即 980 ps, 约为 1020 MIPS。注意 R 型只需 730 ps 却同样被拖到 980 ps, 浪费了约 25% 的时间——这正是把各阶段拆成五级流水线 (下一章) 的直接动机: 拆分后时钟周期可缩短到约 250 ps, 主频提升近四倍。

5.6 本章小结

- 数据通路由 PC、指令存储器、寄存器堆、ALU、数据存储器、符号扩展单元及若干多路选择器构成; 每个部件都有明确的输入输出。
- 单周期执行分为 IF、ID、EX、MEM、WB 五步, 全部在一个时钟周期内完成, $CPI = 1$ 。
- R 型不访存, *lw* 读存储器、*sw* 写存储器、*beq* 用 ALU 相减并改写 PC; 四者共用一条通路, 仅控制信号不同。
- 三个关键 MUX 分别控制 PC 来源、ALU 第二输入、写回数据来源; 控制信号分散作用于各部件。
- 时钟周期由最长关键路径 (*lw*) 决定, 简单指令被迫等待, 效率低——这是流水线的动机。

5.7 常见误区

- **PC 会自动加 4？** 数据通路里**没有**独立的”PC 自增计数器”。PC+4 必须由一个**专用**加法器算出，再经 PC 来源 MUX 送回 PC；即使是 sw/beq 也要计算 PC+4。
- **分支偏移直接用？** beq 的 16 位 offset 是**有**符号数，须先**符号扩展**、再**左移 2 位** ($\times 4$)，且要加到 **PC+4** 而非当前 PC 上。
- **比较用 ALU 结果？** beq 判断相等看的是 ALU 的 **Zero** 标志 ($\$rs-\$rt=0$)，不是 ALU 的数据输出； $\$rs-\rt 的数值本身被丢弃。
- **把两个加法器搞混？** 一个是 PC+4 加法器，另一个是分支目标加法器 ($PC+4 + offset \times 4$)，二者功能不同。
- **写回目标固定？** R 型写 rd，I 型 (lw) 写 rt，靠 RegDst 选择；写回阶段还要靠 MemtoReg 区分”ALU 结果”与”存储器数据”。
- **符号扩展 vs 零扩展：** 地址偏移和算术立即数用**符号扩展**；无符号位运算用**零扩展**，二者不可混用。
- **sw 不需要写回？** 正确。sw 的 RegWrite=0，虽仍有 WB 阶段，但不产生任何寄存器写入。

6 处理器：控制器

6.1 控制器的作用与模型

数据通路只提供了“做什么”的硬件（加法器、寄存器堆、存储器、多路选择器），而“何时做、做哪种运算、结果写到哪里”则由**控制器**（Control Unit）统一指挥。控制器是 CPU 的“大脑”：它按 PC 逐条取出指令，把指令中的操作码（opcode）与功能码（funct）译码，必要时再结合状态标志（如 ALU 的 Zero 标志），在恰当时刻产生一组**控制信号**，驱动数据通路各部件协同工作。

从抽象层次看，控制器的本质是一个**有限状态机**（FSM）：输入是指令字段与状态标志，输出是一束控制信号。记控制信号向量为 $\vec{C}$ ，则

$$\vec{C} = f(\text{opcode}, \text{funct}, \text{状态标志}) \quad (37)$$

其中 opcode 决定指令大类（R 型 / lw / sw / beq 等），funct 只在 R 型指令中进一步区分具体运算。图 28 给出控制器的黑盒模型。

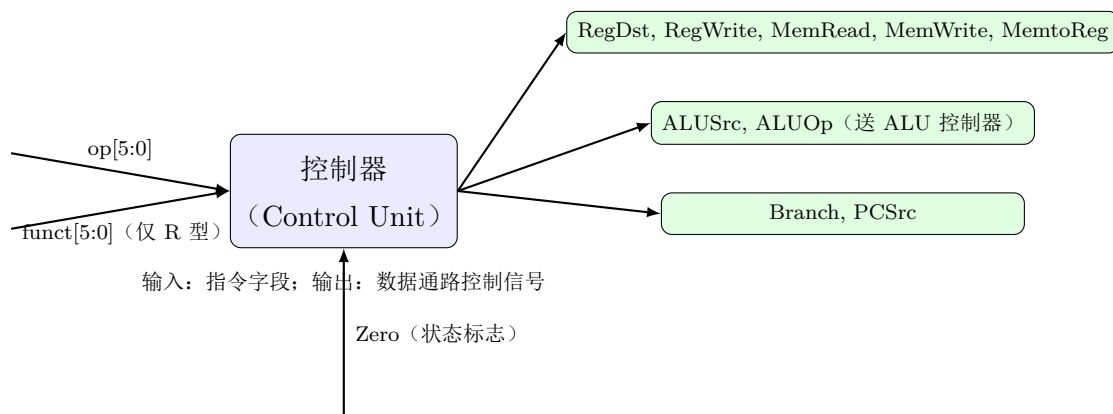


图 28: 控制器的黑盒模型：opcode/funct 译码后产生一束控制信号

控制信号可按作用对象分为三类：**写使能类**（RegWrite、MemWrite）、**读使能类**（MemRead）、**选择类**（RegDst、ALUSrc、MemtoReg、ALUOp）。理解控制器的关键，就是搞清楚“每种指令需要在哪些位置、什么时刻、给哪个信号赋什么值”。

6.2 控制信号的语义与取值

6.2.1 主要控制信号

表 20 汇总了 MIPS 单周期数据通路中的八个主要控制信号。注意区分三种取值语义：**1 bit 使能**（1 有效）、**1 bit 选择**（1 选后者、0 选前者）、**2 bit 编码**（如 ALUOp）。

表 20: MIPS 主要控制信号及其含义

信号	位宽	取值	含义
RegDst	1	0 / 1	写回目标寄存器：0 $\rightarrow$ <i>rt</i> , 1 $\rightarrow$ <i>rd</i>
RegWrite	1	0 / 1	寄存器堆写使能（1 写入）
ALUSrc	1	0 / 1	ALU 第二操作数：0 $\rightarrow$ 寄存器 <i>rt</i> , 1 $\rightarrow$ 符号扩展立即数
Mem-toReg	1	0 / 1	写回数据来源：0 $\rightarrow$ ALU 结果, 1 $\rightarrow$ 存储器读出值
Mem-Read	1	0 / 1	数据存储器读使能
MemWrite	1	0 / 1	数据存储器写使能
Branch	1	0 / 1	条件分支使能，与 Zero 相与得到 PCSrc
ALUOp	2	00 / 01 / 10	ALU 粗操作：00 $\rightarrow$ 加法, 01 $\rightarrow$ 减法, 10 $\rightarrow$ 由 funct 决定

其中 Branch 并不直接改变 PC，而是与 ALU 的比较结果 Zero 相与：

$$\text{PCSrc} = \text{Branch} \cdot \text{Zero} \quad (38)$$

只有当“这是分支指令（Branch=1）”且“两个操作数相等（Zero=1）”时，PC 才被写入分支目标地址。

6.2.2 ALU 控制器：ALUOp 与 funct 到 ALUctr

主控制器只发出粗粒度的 ALUOp；真正的运算选择由 ALU 控制器完成：它在 ALUOp=10 时进一步查看 R 型的 funct 字段，输出细粒度的 3 位 ALUctr。约定 ALUctr 的编码为：000 → AND，001 → OR，010 → ADD，110 → SUB，111 → SLT（set-on-less-than）。

表 21: ALU 控制真值表（ALUOp 与 funct 到 ALUctr）

ALUOp	funct[5:0]	运算	ALUctr[2:0]
00	xxxxxx	加（取地址 / 立即数）	010
01	xxxxxx	减（比较是否相等）	110
10	100000	add	010
10	100010	sub	110
10	100100	and	000
10	100101	or	001
10	101010	slt	111

6.2.3 ALUctr 的化简

对表 21 做逻辑化简。记 ALUOp 的两位为 $ALUOp_1 ALUOp_0$ ，funct 的第 i 位为 f_i （ f_0 为最低位）。观察可提取三个有用的中间量：

- R 型（ $ALUOp_1 = 1$ ）中，只有 and/or 的 $f_2 = 1$ ，其余 $f_2 = 0$ ；
- 需要做减法的指令（sub、slt）恰好 $f_1 = 1$ ；
- 需要产生“1”输出位的指令（or、slt）满足 $f_3 + f_0 = 1$ 。

于是得到

$$\begin{aligned}
 ALUctr_2 &= \overline{ALUOp_1} + \overline{f_2} && \text{（非 R 型恒为 1；R 型看 } f_2 \text{）} \\
 ALUctr_1 &= \overline{ALUOp_1} ALUOp_0 + ALUOp_1 f_1 && \text{（减法/小于置位）} \\
 ALUctr_0 &= ALUOp_1 (f_3 + f_0) && \text{（OR 或 SLT 的最低位置 1）}
 \end{aligned} \tag{39}$$

例如 R 型 or（funct=100101， $f_3 f_2 f_1 f_0 = 0101$ ）代入： $ALUctr_2 = \bar{1} + \bar{1} = 0$ ， $ALUctr_1 = 0 + \underline{1} \cdot f_1 = 0$ ， $ALUctr_0 = 1 \cdot (0 + 1) = 1$ ，得 001，正是 OR。化简至此只需少量门电路即可实现，体现了“把大真值表压成小逻辑”的思想。

6.3 典型指令的控制信号组合

把各类指令横排，把各信号纵排，就得到**指令-信号对照表**（表 22）。它是设计控制器时最重要的一张“答案表”：控制器要做的事，就是让每一种指令都产生它所在列的那一组值。表中 **x** 表示该信号对该指令无关（don't care），实际实现中可任取 0 或 1。

表 22: 四类典型指令的控制信号取值

控制信号	R 型 (add/sub/...)	lw	sw	beq
RegDst	1	0	x	x
ALUSrc	0	1	1	0
MemtoReg	0	1	x	x
RegWrite	1	1	0	0
MemRead	0	1	0	0
MemWrite	0	0	1	0
Branch	0	0	0	1
ALUOp	10	00	00	01

逐行验证一遍即可加深理解：R 型要写 **rd** (RegDst=1)，ALU 用两个寄存器 (ALUSrc=0)，写回 ALU 结果 (MemtoReg=0)；lw 要写 **rt** (RegDst=0)，ALU 用立即数算地址 (ALUSrc=1)，写回存储器数据 (MemtoReg=1)；sw 不写寄存器 (RegWrite=0) 只写存储器 (MemWrite=1)；beq 不做任何读写，只把 ALU 的减法结果送 Zero (ALUOp=01)。

6.4 硬布线控制器

6.4.1 从真值表到布尔表达式

硬布线控制器（Hardwired Control）直接用组合逻辑门实现表 22。做法是：先把 opcode 送译码器产生“最小项”，再用 OR 阵列按布尔式选取需要的项，输出各控制信号。其结构如图 29。

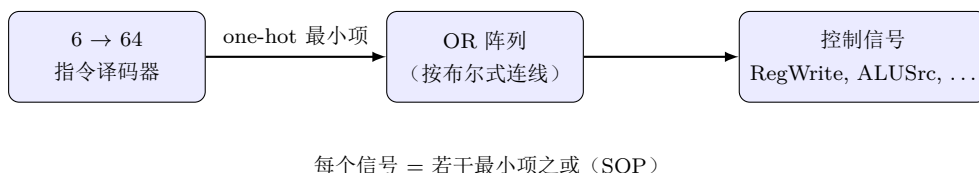


图 29: 硬布线控制器：译码器 + OR 阵列

记 opcode 的第 i 位为 op_i 。四类指令的编码为 R 型 000000、lw 100011、sw 101011、beq 000100。由此可直接写出各信号的布尔表达式，见表 23。

表 23: 控制信号的布尔表达式（未用编码作 don't care）

信号	布尔表达式
RegDst	$\overline{op_5} \overline{op_4} \overline{op_3} \overline{op_2} \overline{op_1} \overline{op_0}$ (R 型)
RegWrite	$\text{RegDst} + (\text{lw} = op_5 \overline{op_4} \overline{op_3} \overline{op_2} op_1 op_0)$
ALUSrc	$op_5 \overline{op_4} op_1 op_0$
MemRead	$\text{lw} = op_5 \overline{op_4} \overline{op_3} \overline{op_2} op_1 op_0$
MemWrite	$\text{sw} = op_5 \overline{op_4} op_3 \overline{op_2} op_1 op_0$
MemtoReg	MemRead
Branch	$\text{beq} = \overline{op_5} \overline{op_4} \overline{op_3} op_2 \overline{op_1} \overline{op_0}$

6.4.2 化简例题

示例 6.1 (化简 ALUSrc). $ALUSrc$ 只在 lw (100011) 与 sw (101011) 上为 1。先写成最小项之和：

$$ALUSrc = op_5 \overline{op_4} \overline{op_3} \overline{op_2} op_1 op_0 + op_5 \overline{op_4} op_3 \overline{op_2} op_1 op_0.$$

两个最小项仅在 op_3 上不同（一个取 0、一个取 1），其余位完全相同，故 op_3 可吸收消去；再把公共因子提出：

$$ALUSrc = op_5 \overline{op_4} \overline{op_2} op_1 op_0 (\overline{op_3} + op_3) = op_5 \overline{op_4} \overline{op_2} op_1 op_0.$$

若把未使用编码当作 don't care，还可进一步去掉恒为 0 的 $\overline{op_2}$ ，得到表 23 中更简的形式 $op_5 \overline{op_4} op_1 op_0$ 。

示例 6.2 (化简 $RegWrite$). $RegWrite$ 在 R 型 (000000) 与 lw (100011) 上为 1, 共两个最小项。两者共享因子 $\overline{op_4} \overline{op_3} \overline{op_2}$, 剩下部分对 R 型是 $\overline{op_5} \overline{op_1} \overline{op_0}$ 、对 lw 是 $op_5 op_1 op_0$, 故

$$RegWrite = \overline{op_4} \overline{op_3} \overline{op_2} (\overline{op_5} \overline{op_1} \overline{op_0} + op_5 op_1 op_0).$$

也可记作 “ $RegWrite = R \text{ 型} + lw$ ”, 即用 R 型译码线或 lw 译码线直接驱动, 这正是译码器 + **OR** 阵列的直观含义。

硬布线的优点是速度快 (信号在一个门延迟链路内产生), 缺点是电路与指令集强耦合: 一旦增加或修改指令, 就要重新综合整片逻辑, 灵活性差、设计周期长。因此它多用于指令规整精简的 RISC 处理器。

6.5 微程序控制器

6.5.1 微指令格式与控制存储器

微程序控制器 (Microprogrammed Control) 换一种思路: 把每条机器指令的执行过程拆成若干步, 每一步对应一条微指令, 微指令的每一位直接决定一根控制信号。所有微指令预先存放在控制存储器 (Control Memory, 通常是 ROM) 中, 机器指令的动作序列即一段微程序。

一条微指令通常由两部分组成 (图 30): 微操作控制字段 (每一位对应一根控制信号, 1 表示该信号有效) 与顺序控制字段 (给出下一条微指令的地址, 或指明如何根据条件判别下地址)。



图 30: 微指令格式: 控制字段 + 顺序控制字段

6.5.2 微程序的执行

微程序执行依赖一个微程序计数器 (μPC) 和微地址形成逻辑：

1. 取机器指令，用其 opcode 查一张微程序入口表，得到该指令微程序的首地址，送入 μPC ；
2. 从控制存储器取出 μPC 指向的微指令，其控制字段驱动数据通路完成一步微操作；
3. 由顺序控制字段与状态标志计算下一条微指令地址，写回 μPC ；
4. 重复步骤 2-3，直到遇到“取指微指令”的结束标志，回到步骤 1。

表 24 给出 `lw` 指令的一段微程序示意（与多周期的状态划分一致，详见下一节）。

表 24: `lw` 指令的微程序片段（示意）

微地址	阶段	微操作（有效控制信号）
00	取指	$\text{IR} \leftarrow \text{M}[\text{PC}]$, $\text{PC} \leftarrow \text{PC}+4$ (MemRead, IRWrite, PCWrite)
01	译码	$\text{A} \leftarrow \text{Reg}[\text{rs}]$, $\text{B} \leftarrow \text{Reg}[\text{rt}]$ (读寄存器堆)
10	执行	$\text{ALUOut} \leftarrow \text{A} + \text{SignExt}(\text{imm})$ (ALUSrc=1, ALUOp=00)
11	访存	$\text{MDR} \leftarrow \text{M}[\text{ALUOut}]$ (MemRead)
100	写回	$\text{Reg}[\text{rt}] \leftarrow \text{MDR}$ (RegWrite=1, MemtoReg=1)

6.5.3 与硬布线的对比

表 25: 硬布线控制器与微程序控制器对比

对比项	硬布线	微程序
实现介质	组合逻辑门	控制存储器 (ROM)
控制信号来源	布尔表达式的门输出	微指令控制字段的位
速度	快 (一个门延迟)	较慢 (需读控制存储器的周期)
灵活性	差, 改指令即改电路	好, 改微程序即可
规整性	电路庞杂	规整、易调试
典型架构	RISC (如 MIPS)	CISC (如 x86)

一句话概括：硬布线把控制“写死在电路里”，微程序把控制“写进存储器里”。

6.6 单周期与多周期 CPU

6.6.1 状态划分

单周期 CPU 把取指、译码、执行、访存、写回全部塞进一个时钟周期；多周期 CPU 则把它们拆成若干状态，每个状态占一个时钟周期。以 MIPS 为例，可划分五个状态：

- **IF**（取指）： $IR \leftarrow M[PC]$, $PC \leftarrow PC+4$;
- **ID**（译码 / 读寄存器）：读 $Reg[rs]$ 、 $Reg[rt]$ ，并计算分支目标 $ALUOut \leftarrow PC + (SignExt(imm) \ll 2)$;
- **EX**（执行 / 地址计算）：R 型做运算，lw/sw 算地址，beq 做比较；
- **MEM**（访存）：lw 读、sw 写；
- **WB**（写回）：R 型写 rd，lw 写 rt。

不同指令经过的状态数不同：beq 只需 3 个（IF、ID、EX），R 型需 4 个（IF、ID、EX、WB），lw 需 5 个（IF、ID、EX、MEM、WB），sw 需 4 个（IF、ID、EX、MEM）。状态转移如图 31。

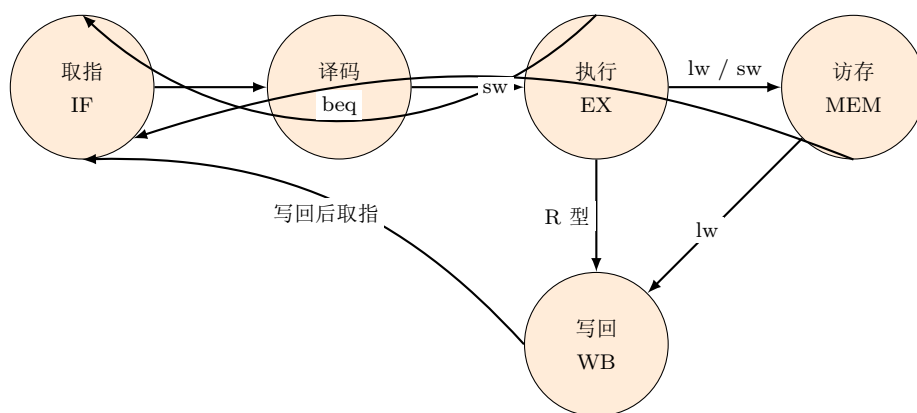


图 31: 多周期控制器的状态机（不同指令走不同长度的路径）

6.6.2 各状态的控制信号

把每个状态“激活”的控制信号列出，即为多周期控制器的核心设计表（表 26）。与单周期不同，同一根信号（如 ALUSrc、RegWrite）在不同状态取不同值，控制信号随状态推进而变化。

表 26: 多周期各状态的有效控制信号（简化）

状态	主要有效控制信号
IF	MemRead=1, IRWrite=1, PCWrite=1, ALUSrcA=0, ALUSrcB=01, ALUOp=00
ID	ALUSrcA=0, ALUSrcB=11（立即数左移 2）, ALUOp=00（算分支目标）
EX	R 型: ALUSrcA=1, ALUSrcB=00, ALUOp=10; lw/sw: ALUSrcB=10, ALUOp=00; beq: ALUOp=01
MEM	lwrD=1, lw 置 MemRead=1 / sw 置 MemWrite=1
WB	RegWrite=1; R 型 RegDst=1, MemtoReg=0; lw RegDst=0, MemtoReg=1

6.6.3 为什么多周期的时钟周期更短

单周期的时钟周期必须容纳最慢指令的最长路径（通常是最长的 lw）:

$$T_{\text{single}} \approx T_{\text{IF}} + T_{\text{ID}} + T_{\text{EX}} + T_{\text{MEM}} + T_{\text{WB}}, \quad T_{\text{multi}} \approx \max\{T_{\text{IF}}, T_{\text{ID}}, T_{\text{EX}}, T_{\text{MEM}}, T_{\text{WB}}\}. \quad (40)$$

多周期把一个长周期切成若干个短周期，每个周期只走一段路径，时钟周期约为最长阶段的耗时（常为访存或 ALU 阶段），因此周期可以做得短得多。代价是 CPI 从 1 升到 3-5，且需要额外的寄存器（A、B、ALUOut、IR、MDR）保存中间结果。

表 27: 单周期与多周期对比

特征	单周期	多周期
CPI	1	3-5（视指令而定）
时钟周期	由最慢指令决定	由最慢阶段决定（更短）
硬件重复	有（多个加法器等）	少（组件可复用）
中间寄存器	无	需要（A、B、ALUOut 等）
控制器复杂度	低	中

多周期是单周期通往流水线的中间形态：它实现了组件复用与短周期，但仍有功能

单元在多数周期里闲置。下一章的流水线，正是在“短周期 + 状态重叠”的基础上把吞吐率推到极限。

6.7 本章小结

- 控制器的职责：依据 opcode/funct（及状态标志）产生控制信号，是数据通路的“指挥官”。
- 八个主要控制信号（RegDst、ALUSrc、MemtoReg、RegWrite、MemRead、MemWrite、Branch、ALUOp）决定了数据的来源、去向与写使能。
- ALU 控制分两级：主控制器给粗粒度 ALUOp，ALU 控制器结合 funct 产生 ALUctr；可用布尔式化简（式 39）。
- 硬布线用组合逻辑实现，快但不灵活；微程序把控制存入 ROM，灵活但较慢。
- 多周期通过拆分状态缩短时钟周期并复用组件，是通向流水线的桥梁。

6.8 常见误区

- 把 RegWrite 当作“写 rd”：RegWrite 只是写使能，写哪个寄存器由 RegDst 选（rt 还是 rd），二者不能混淆。
- 认为 Branch 直接改 PC：Branch 只是使能，必须与 ALU 的 Zero 相与得到 PCSrc（式 38），beq 不相等时 PC 仍取 PC+4。
- 给 sw 也置 RegWrite：sw 不写寄存器堆（RegWrite=0），只写存储器（MemWrite=1）。
- 把 ALUOp 与 ALUctr 混为一谈：ALUOp 是 2 位粗控制，ALUctr 是 3 位细控制；只有 R 型才需要看 funct。
- 误以为多周期一定更快：多周期缩短的是时钟周期，但 CPI 增大；总时间未必优于精心设计的单周期，真正的大幅提升要靠流水线。

7 流水线与指令级并行

7.1 流水线思想与时空图

单个组合电路完成一条指令的全部工作，需要依次经过取指、译码、执行、访存、写回等若干步骤。若让每条指令串行走完全部步骤，则在任意时刻电路中只有一小部分在被使用，大量硬件处于空闲。**流水线**（Pipeline）的核心思想，就是把一条指令的执行切分为若干个延时相当的功能段，让不同指令的**不同功能段在时间上重叠执行**——正如工厂装配线上，每个工位（功能段）在同一时刻处理着不同产品（指令）的不同工序。

理解流水线最直观的工具是**时空图**（Space-Time Diagram）：横轴为时间（时钟周期），纵轴为功能段，一条指令的推进在图上表现为一条自左下到右上的斜线。

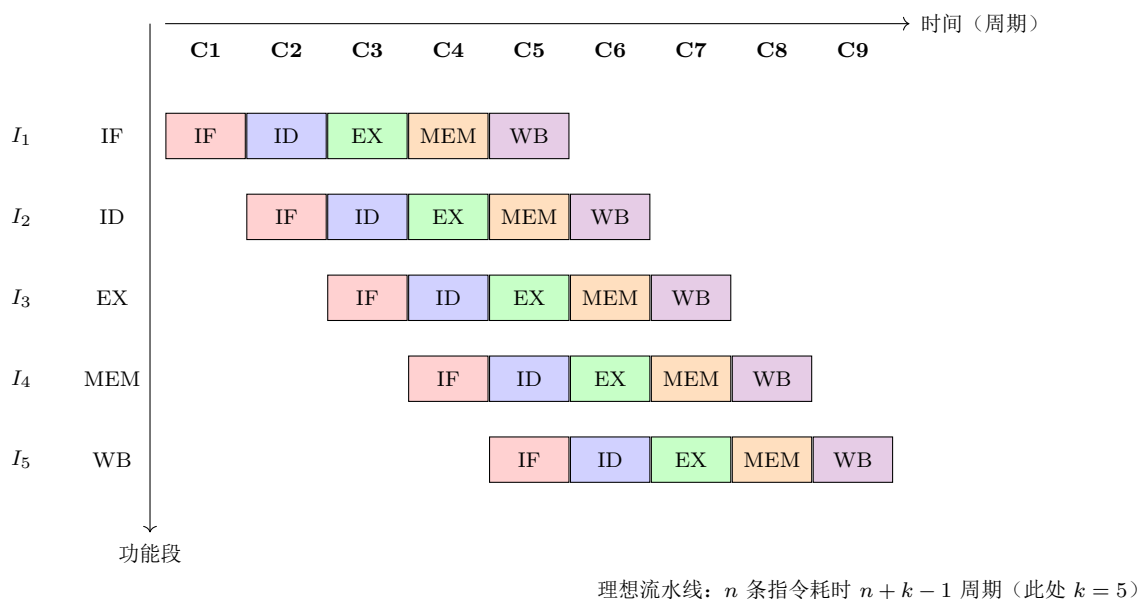


图 32: 五级流水线时空图（无冒险）：填充 $k - 1$ 个周期后每周期流出一条结果

图 32 揭示了两点关键事实：

- 流水线存在 $k - 1$ 个周期的**填充（fill）**阶段，此阶段尚未满载；
- 进入稳定状态后，**每个时钟周期都有一条指令完成**，吞吐率不再随指令数增长。

7.2 理想加速比与吞吐率

为定量刻画，设流水线有 k 级，每级延时为 T_s ，稳态下时钟周期 $T_{\text{clk}} = T_s$ （各级已平衡）。执行 n 条指令时：

$$T_{\text{串行}} = n \cdot k \cdot T_s \quad (41)$$

即每条指令都完整地穿过 k 级，共需 nk 个段延时。

$$T_{\text{流水}} = (k + (n - 1)) \cdot T_s = (n + k - 1) \cdot T_s \quad (42)$$

其中 k 个周期用于填充，此后每周期完成一条，完成余下 $n - 1$ 条需 $n - 1$ 个周期。

$$S = \frac{T_{\text{串行}}}{T_{\text{流水}}} = \frac{nkT_s}{(n + k - 1)T_s} = \frac{nk}{n + k - 1} \xrightarrow{n \gg k} k \quad (43)$$

由式 (43) 可见，当指令数远大于级数时，理想加速比趋于流水线级数 k 。吞吐率（单位时间完成的指令数）为

$$\text{TP} = \frac{n}{T_{\text{流水}}} = \frac{n}{(n + k - 1)T_s} \xrightarrow{n \gg k} \frac{1}{T_s} = \frac{1}{T_{\text{clk}}}, \quad (44)$$

即稳态吞吐率的理论上限是每周期一条指令（IPC = 1）。

示例 7.1 (理想加速比与吞吐率). 取 $k = 5$ 级流水线，每级延时 $T_s = 1 \text{ ns}$ ($T_{\text{clk}} = 1 \text{ ns}$)，执行 $n = 1000$ 条指令。

- 串行： $T_{\text{串行}} = nkT_s = 1000 \times 5 \times 1 = 5000 \text{ ns}$ ，吞吐率 $1000/5000 = 0.2 \text{ 条/ns}$ 。
- 流水： $T_{\text{流水}} = (n + k - 1)T_s = (1000 + 5 - 1) \times 1 = 1004 \text{ ns}$ ，吞吐率 $1000/1004 \approx 0.996 \text{ 条/ns}$ 。
- 加速比 $S = 5000/1004 \approx 4.98$ ，非常接近理论上限 $k = 5$ （差距来自 $k - 1 = 4$ 个填充周期，仅占约 0.4%）。

若 n 缩小到 20，则 $T_{\text{流水}} = (20 + 4) = 24 \text{ ns}$ ， $S = 100/24 \approx 4.17$ ——指令越少，填充开销占比越大，加速比越远离 k 。

7.3 经典五级流水线

7.3.1 五个阶段

MIPS 类 RISC 处理器把指令执行划分为五个延时均衡的阶段（表 28）。

表 28: 五级流水线各阶段的功能

阶段	全称	主要功能
IF	Instruction Fetch	以 PC 为地址取出指令，并计算 $PC + 4$
ID	Instruction Decode	译码、读寄存器堆、生成立即数、识别分支
EX	Execute	ALU 运算、分支地址/条件判断、访存地址计算
MEM	Memory Access	读/写数据存储器（仅 load/store 做实际访存）
WB	Write Back	将结果写回寄存器堆（仅需写回的指令）

一条典型的 `lw` 指令在其存在的每个周期只占用对应的一段硬件，其余四段硬件同时服务于其他指令，这正是流水线提升资源利用率的方式。

7.3.2 流水线寄存器的作用

要让不同阶段“各自干各自的”，必须把每一级的中间结果锁存下来，交给下一级在下一个时钟周期使用。连接相邻两级之间的小型寄存器称为**流水线寄存器**（Pipeline Register），它们受同一时钟驱动，是流水线的“传送带”（表 29）。

表 29: 四条流水线寄存器及其携带的信息

流水线寄存器	传递的内容
IF/ID	取出的指令字、 $PC + 4$ （供后续分支与 <code>jal</code> 使用）
ID/EX	读出的两个寄存器值、立即数、目标寄存器号、已译码的控制信号
EX/MEM	ALU 结果、待写入存储器的数据、目标寄存器号、存储器/写回控制信号
MEM/WB	访存读出的数据或 ALU 结果、目标寄存器号、写回控制信号

流水线寄存器带来的代价是延迟增大（每级多一次寄存器的建立/保持时间），收益是电路可以被安全地划分到不同周期。它也是理解冒险的钥匙：数据必须“写入”一个寄存器后，才可能在下一周期被别的阶段读到。

7.3.3 时空图

结合图 32, I_1 的 WB 与 I_5 的 IF 发生在同一周期 (C5)。在满载区段, 每个周期的边界恰好有 {1 IF, 1 ID, 1 EX, 1 MEM, 1 WB} 各一次, 硬件利用率达到 100% (忽略冒险)。

7.4 流水线冒险

理想流水线要求“每条指令都能按时进入下一级”, 但**冒险** (Hazard) 会打破这一假设。冒险分三类: **结构冒险** (资源冲突)、**数据冒险** (数据依赖)、**控制冒险** (控制转移)。

7.4.1 结构冒险

当两条指令在同一周期争用同一硬件资源时, 发生结构冒险。例如经典的“单存储器”设计里, IF 要取指令、MEM 要读写数据, 二者同时访问存储器冲突。常用解决手段见表 30。

表 30: 结构冒险的常见来源与对策

冲突来源	对策
取指与访存争用存储器	分离指令 cache 与数据 cache (哈佛结构)
写回与译码争用寄存器堆	前半个周期写、后半个周期读, 或增设写端口
功能单元未流水化	将乘除法器改为流水线化, 或增加多份功能单元

7.4.2 数据冒险与数据前递

当某条指令要用的数据还没被前一条指令产生或写回时, 就出现数据冒险。其中最常见的是**先写后读** (Read After Write, RAW):

示例 7.2 (RAW 数据冒险).

```

1 add $1, $2, $3    # $1 在 WB 阶段才写入寄存器堆
2 sub $4, $1, $5    # 但 sub 在 ID 阶段就要读 $1 —— 读到的是旧值, 冒险!
```

若不处理, *sub* 会读到过时的 *\$1*, 程序结果错误。

朴素的做法是插气泡等待, 但等待很浪费。观察时空图可以发现: *add* 的 ALU 结果在 **EX** 结束时已算出, *sub* 的 EX 阶段要到下一个周期才开始——两者正好差一个周期, 结果可以在产生点直接被“旁路”过去。这就是**前递** (Forwarding, 也叫旁路 Bypassing), 如图 33 所示。

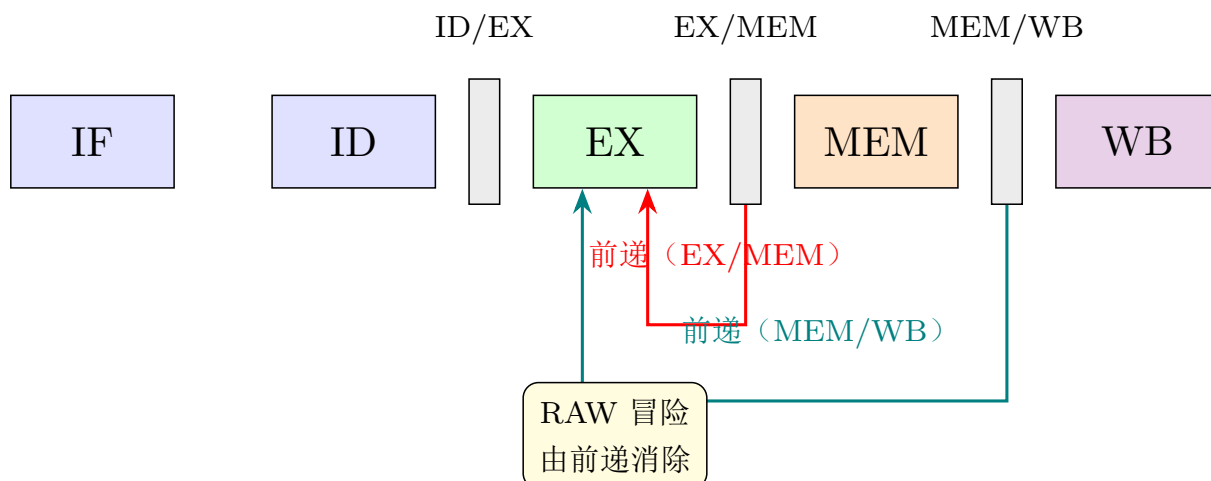


图 33: 数据前递: 把 EX/MEM、MEM/WB 中已算出的结果直接旁路到 EX 阶段输入

前递并非万能。若生产数据的指令是 `lw`, 其数据要到 MEM 阶段末才可用, 而消费指令的 EX 紧接其后——前递也来不及, 此时必须插入一个气泡 (见 7.5 小节)。更一般地, 流水线需要前递单元在 EX 入口处比较“源寄存器号”与“EX/MEM、MEM/WB 中的目标寄存器号”, 若匹配则选择前递数据而非寄存器堆数据。

7.4.3 控制冒险

分支、跳转等控制转移指令会改变 PC, 而流水线在分支结果算出之前就已按顺序取入了后续指令 (对 `beq` 而言, 分支是否发生要到 EX 阶段才确定)。若分支真的跳转, 已取入的指令就是错误的, 必须作废, 这就是控制冒险。控制冒险造成的损失称为分支惩罚 (Branch Penalty), 其大小取决于“何时知道分支去向”和“预测是否正确”。

7.5 停顿、分支惩罚与分支预测

7.5.1 停顿与分支延迟槽

停顿 (Stall) 是最简单的应对：让相关指令或后续取指暂停若干周期，向流水线中插入不产生副作用的**气泡** (Bubble，常以“清空控制信号”的空操作实现)。分支引起的停顿周期数即惩罚；一个常见的折中是**分支延迟槽** (Branch Delay Slot)：无论分支是否 jump，紧随分支之后的那一条指令都照常执行，编译器把一条“一定会执行”的指令填入该槽，从而把惩罚从 2 个周期降到 1 个周期。

7.5.2 静态预测

静态预测在编译或上电时固定策略，硬件简单：

- **总是预测跳转** (always taken)：适合循环回边等高频跳转，但每次循环退出都错一次；
- **总是预测不跳转** (always not taken)：实现最简单（顺序取指即可），适合“分支大多不跳”的代码。

7.5.3 动态预测

动态预测在运行时依据历史记录预测，准确率更高。最基础的是 **1 位预测器**：用 1 bit 记录该分支“上次跳/未跳”，本次就预测相同结果。它对循环的“边界抖动”敏感——每次循环退出都会连错两次。改进方案是 **2 位饱和计数器**（2-bit Saturating Counter）：计数器的增减需要连续两次相反结果才翻转预测，从而过滤单次抖动，如图 34 所示。

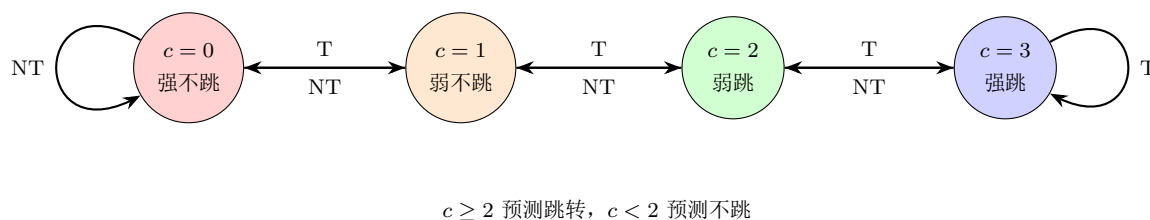


图 34: 2 位饱和计数器分支预测状态机：T 跳、NT 不跳

2 位预测器的更新规则可用算法 1 描述。

Algorithm 1 两位饱和计数器分支预测

```

1: 对每条分支维护计数器  $c \in \{0, 1, 2, 3\}$ ，初值  $c \leftarrow 1$ 
2: for 每条分支指令  $b$  do
3:   if  $c \geq 2$  then
4:     预测结果  $\leftarrow$  跳转
5:   else
6:     预测结果  $\leftarrow$  不跳转
7:   end if
8:   执行后得到真实结果  $r$ 
9:   if  $r = \text{跳转}$  then
10:     $c \leftarrow \min(3, c + 1)$ 
11:   else
12:     $c \leftarrow \max(0, c - 1)$ 
13:   end if
14: end for

```

示例 7.3 (分支预测准确率与误预测代价). 某程序共 $n = 10^4$ 条指令，其中分支指令占 20%，即 2000 条；每次误预测的惩罚为 2 个周期。三种预测器的准确率与误预测次数如下表：

预测器	准确率	误预测次数	额外停顿周期
静态（总是不跳）	65%	700	$700 \times 2 = 1400$
1 位动态预测器	82%	360	$360 \times 2 = 720$
2 位饱和计数器	90%	200	$200 \times 2 = 400$

误预测次数 = 分支总数 $\times (1 - \text{准确率})$ ，例如 2 位预测器为 $2000 \times 10\% = 200$ 。可见准确率每提高一点，都能显著减少昂贵的停顿周期——这正是现代处理器把分支预测器做得越来越复杂的原因。

7.6 流水线性能分析

引入停顿后，理想“每周期一条”不再成立。设平均每条指令因各类冒险额外停顿 s 个周期（stalls per instruction），则

$$\text{CPI} = 1 + s, \quad T = n \cdot \text{CPI} \cdot T_{\text{clk}} = n \cdot (1 + s) \cdot T_{\text{clk}}. \quad (45)$$

式 (45) 与式 (43) 联合给出含停顿的实际加速比：

$$S = \frac{nkT_s}{n(1+s)T_s} = \frac{k}{1+s} \quad (n \gg k). \quad (46)$$

可见实际加速比总是被因子 $1/(1+s)$ 拉低； $s = 0$ 时才恢复到理想值 k 。等效地，含停顿的吞吐率为

$$\text{TP} = \frac{n}{T} = \frac{1}{(1+s)T_{\text{clk}}} \quad (\text{条/周期}). \quad (47)$$

需要提醒的是，若以“有效率” $\eta = 1/(1+s)$ 表示流水线被冒险浪费的比例，则个别教材也把上式等价地写成 $T = \frac{nT_{\text{clk}}}{1 + \text{stalls}}$ 的形式来强调吞吐率下降，本质相同，只是记号约定不同。

示例 7.4 (含停顿的加速比). 沿用 $k = 5$ 、 $T_{\text{clk}} = 1 \text{ ns}$ 、 $n = 10^4$ 条指令，采用 2 位预测器（额外停顿 400 周期）。

- 理想周期数（含填充） $C_{\text{ideal}} = n + k - 1 = 10004$ ，串行时间 $T_{\text{串行}} = nkT_s = 50000 \text{ ns}$ 。
- 理想加速比 $S_{\text{ideal}} = 50000/10004 \approx 5.00$ 。
- 含停顿周期数 $C = 10004 + 400 = 10404$ ， $T = 10404 \text{ ns}$ 。
- 实际加速比 $S = 50000/10404 \approx 4.81$ 。
- 校验： $\text{CPI} = 10404/10^4 = 1.04$ ，故 $s = 0.04$ ；由式 (46)， $S \approx 5/1.04 \approx 4.81$ ，与逐周期计算一致。

7.7 超标量与乱序执行

单发射流水线每周期最多一条指令 ($IPC \leq 1$)。突破这一上限需要更深层的指令级并行技术。

表 31: 从标量流水线到多发射技术的演进

技术	每周期发射	要点
标量流水线	1	$IPC \leq 1$, 顺序发射、顺序完成
超标量 (Superscalar)	$m > 1$	每周期取/译码/发射多条, 需多个功能单元并行
乱序执行 (OoO)	动态	按数据就绪而非程序顺序执行, 乱序完成、顺序提交
同时多线程 (SMT)	动态	一个核心交替执行多个线程的指令, 隐藏访存延迟

超标量通过复制数据通路、每周期发射多条互不冲突的指令来提升 IPC (现代处理器常为 2 ~ 6 发射)。**乱序执行**则放松“按程序顺序执行”的约束: 用**重排序缓冲区** (Reorder Buffer, ROB) 暂存已执行但尚未提交的结果, 用**寄存器重命名** (Register Renaming) 消除 WAR、WAW 等假依赖, 只保留真正的数据依赖, 最后仍按程序顺序**提交** (Commit) 以保证精确异常。

一句话概括演进逻辑: 流水线用**时间重叠**换取吞吐率; 超标量在此之上用**空间复制**; 乱序执行再用**动态调度**填满被依赖链和访存延迟“挖出”的空洞。

7.8 本章小结与常见误区

本章从流水线思想出发，给出了理想加速比 $S \approx k$ 与吞吐率上限 $1/T_{\text{clk}}$ ，剖析了五级流水线的 IF/ID/EX/MEM/WB 结构、流水线寄存器的锁存作用，以及结构冒险、数据冒险（RAW 与前递）、控制冒险（死区、分支惩罚、延迟槽、静态/动态预测）三类问题，最后给出含停顿的性能模型 $\text{CPI} = 1 + s$ （式 (45)）与超标量、乱序执行的进阶方向。

常见误区：

- 把理想加速比当成实际加速比。 $S \approx k$ 只在无冒险、指令数 $n \gg k$ 时成立；考虑填充、停顿后应使用 $S = k/(1 + s)$ ，实际加速比通常明显小于 k 。
- 认为流水线级数越多越好。级数增加会使填充开销、冒险概率、流水线寄存器延迟与时钟偏斜上升，存在收益递减，甚至因 T_{clk} 无法继续减小而得不偿失。
- 以为前递能解决一切 RAW。1w 后立即使用其结果时，数据可用时刻晚于消费指令 EX 的起点，仍须插入一个气泡（load-use 冒险）。
- 混淆吞吐率与延迟。流水线提升的是吞吐率，单条指令的延迟反而因经过寄存器而略微增加。
- 忽视异常与分支的后果。只有把乱序执行的结果按序提交，才能保证分支误预测和异常发生时现场可以精确回滚。

8 存储层次与缓存

8.1 存储层次概述

存储器的速度、容量和价格三者存在矛盾：容量越大越慢越便宜。解决方案是存储层次（Memory Hierarchy）：

$$\text{寄存器} \ll \text{L1 缓存} \ll \text{L2 缓存} \ll \text{L3 缓存} \ll \text{主存} \ll \text{磁盘/SSD} \quad (48)$$

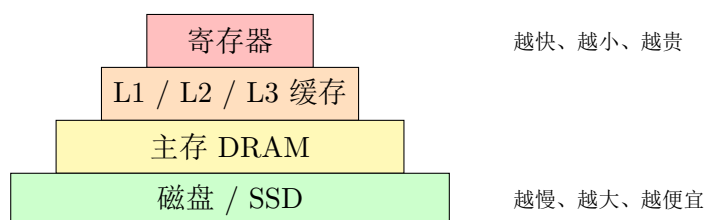


图 35: 存储层次金字塔（利用局部性弥补速度差）

8.2 局部性原理

定义 8.1 (时间局部性). 刚访问过的数据很可能在不久的将来再次被访问（如循环中的 *sum* 变量）。

定义 8.2 (空间局部性). 如果访问了地址 x ，附近的地址 $(x + 1, x + 2, \dots)$ 很可能很快被访问（如顺序数组遍历）。

缓存就是利用这两个局部性，把当前“用得着”的数据从慢速存储器拷贝到快速存储器。

8.3 SDRAM 与 DRAM

- **DRAM**（动态随机存取存储器）：存储单元为电容，需要定期刷新。密度大但慢。HBM（高带宽存储器）是 DRAM 的 3D 堆叠变体。
- **SRAM**（静态随机存取存储器）：6 个晶体管构成双稳态电路，速度快但面积大。用作缓存（Cache）。

8.4 缓存 (Cache)

缓存是 CPU 和主存之间的高速小容量缓冲。现代 CPU 通常有 L1、L2、L3 三级缓存。

8.4.1 直接映射缓存 (Direct-Mapped)

$$\text{缓存行索引} = (\text{块地址}) \bmod (\text{缓存行数}) \quad (49)$$

每个内存块只能映射到唯一一个缓存位置。硬件简单但冲突缺失多。

8.4.2 组相联缓存 (Set-Associative)

$$\text{组索引} = (\text{块地址}) \bmod (\text{组数}) \quad (50)$$

每组有多路 (通常 2-16 路)。是直接映射与全相联的折中。

8.4.3 缓存地址划分

$$\text{地址} = [\text{Tag} \mid \text{Index} \mid \text{Block Offset}]$$

- Block Offset: 块内字节偏移
- Index: 选择缓存行 (直接映射) 或缓存组 (组相联)
- Tag: 验证该缓存行是否存的是所需数据



$$\text{地址} = [\text{Tag} \mid \text{Index} \mid \text{Block Offset}]$$

图 36: 缓存地址划分 (Tag / Index / Block Offset)

8.4.4 缓存缺失与写策略

表 32: 缓存写策略

策略	写命中	写缺失
写穿透 (Write-Through)	同时更新缓存和主存	直接写主存
写回 (Write-Back)	只更新缓存, 标记 dirty	先分配缓存行, 再写入
分配 (Write-Allocate)	—	分配缓存行 (常配合写回)
非分配 (No-Write-Allocate)	—	不分配, 直接写主存

8.4.5 缓存性能

平均访存时间 = 命中时间 + 缺失率 × 缺失代价

$$AMAT = T_{\text{hit}} + \text{Miss Rate} \times \text{Miss Penalty}$$

8.5 虚拟内存

虚拟内存使每个进程拥有独立的地址空间, 且程序可用的地址可以超过物理内存容量。

定义 8.3 (虚拟地址与物理地址).

$$\text{虚拟地址} \xrightarrow{MMU + TLB} \text{物理地址}$$

MMU (*Memory Management Unit*) 负责转换, 页表 (*Page Table*) 存储映射关系。

8.5.1 TLB (快表)

TLB (*Translation Lookaside Buffer*) 是页表的专用缓存, 缓存最近使用的虚拟 → 物理地址映射。TLB 缺失的代价非常高 (需要走访页表, 甚至引起缺页中断)。

8.5.2 多级页表

现代操作系统使用多级页表减少页表本身的内存开销 (如 x86-64 的 4 级页表)。

8.6 存储层次设计原则

- 越靠近 CPU 的层次，容量越小、速度越快、单价越贵
- 包含性 (Inclusion): L1 数据也在 L2 和 L3 中 (多数实现)
- 排他性 (Exclusion): 不同层次不重叠数据 (AMD 的缓存策略)
- 一致性 (Coherence): 多核共享数据时保证所有核看到的值一致

9 I/O 系统与总线

9.1 I/O 系统的层次

输入/输出 (I/O) 系统负责 CPU 与外部世界的通信:

- 设备层: 键盘、鼠标、显示器、硬盘、网卡、GPU
- 设备控制器: 硬件接口, 连接设备和总线
- 设备驱动: 操作系统软件, 向 OS 提供统一接口
- I/O 指令 / 内存映射 I/O: CPU 如何与控制器交互

9.2 内存映射 I/O vs 独立 I/O

表 33: 两种 I/O 模型

特征	内存映射 I/O	独立 I/O
地址空间	I/O 设备与内存共用地址	I/O 有独立地址空间
访问方式	lw/sw 指令	专用 IN/OUT 指令
代表	MIPS、ARM、RISC-V	x86 (in / out)
设计	简单, 无需额外指令	地址不冲突, 但增加复杂度

9.3 I/O 同步方式

9.3.1 轮询 (Polling)

CPU 反复读取设备状态寄存器, 直到设备就绪。简单但浪费 CPU 时间。

```
while (status_reg != READY) { /* 等待 */ };
```

9.3.2 中断 (Interrupt)

设备就绪时向 CPU 发出中断信号。CPU 暂停当前执行, 跳转到中断服务程序 (ISR), 处理完毕后恢复。

- 中断向量表: 存储各中断号对应的 ISR 地址
- 中断优先级: 高优先级中断可打断低优先级
- 可屏蔽中断 vs 不可屏蔽中断 (NMI)

9.3.3 直接存储器访问（DMA）

DMA 控制器直接控制总线，在设备和内存之间搬移数据，不经过 CPU。适用于大批量数据传输（如磁盘读写、网络包）。

- CPU 配置 DMA 控制器（源地址、目标地址、传输大小）
- DMA 独立完成传输
- 完成后向 CPU 发中断通知

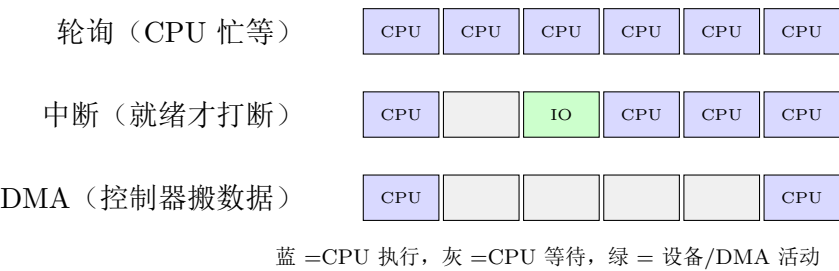


图 37: 轮询、中断与 DMA 的 CPU 时间占用对比

9.4 总线

总线（Bus）是连接 CPU、内存和 I/O 设备的共享通信通道。

9.4.1 总线分类

表 34: 总线类型		
总线	连接对象	代表
处理器-内存总线	CPU ↔ 内存	前-总线（FSB）、QPI、Infinity Fabric
I/O 总线	CPU ↔ 外围设备	PCI、PCIe、USB
背板总线	模块间互连	VME、CompactPCI

9.4.2 现代总线：PCIe

PCI Express (PCIe) 取代了传统并行总线 (PCI)，采用高速串行点对点连接：

- **Lane**：每 Lane 由一对差分信号（发送 + 接收）组成
- **带宽**：PCIe 3.0 $\times 1 \approx 1$ GB/s, $\times 16 \approx 16$ GB/s
- **拓扑**：Root Complex $\rightarrow$ Switch $\rightarrow$ Endpoint，树形结构
- **热插拔、链路协商、错误检测与重传**

9.5 磁盘与固态存储

- **HDD（机械硬盘）**：旋转盘片 + 磁头臂，寻道时间 + 旋转延迟 + 传输时间
- **SSD（固态硬盘）**：NAND Flash，无机械部件，随机读快、顺序读写快
- **NVMe**：基于 PCIe 的快速存储协议（取代 SATA/AHCI），延迟 $< 10 \mu s$

9.6 多核与并行 I/O

现代多核系统面临 I/O 瓶颈：

- **DMA 竞争**：多个核心共享 DMA 通道和内存带宽
- **IOMMU**：用类似 MMU 的机制保护设备对内存的访问
- **RDMA**：远程直接内存访问，跨网络访问远端内存（InfiniBand、RoCE）
- **DPDK / SPDK**：绕过内核，用户态直接操作 I/O，降低延迟

9.7 系统总线架构总结

现代计算机的总线结构已经从” 共享总线” 演变为” 互连网络”:

$\text{CPU} \xleftrightarrow{\text{QPI} / \text{UPI} / \text{IF}} \text{内存控制器} \rightarrow \text{DDR5}$; $\text{CPU} \xleftrightarrow{\text{PCIe}} \text{GPU} / \text{NVMe} / \text{NIC}$

这种点对点互联取代了传统共享总线，提供了更好的带宽隔离和可扩展性。